

UNIVERSIDAD DE GRANADA

TRABAJO FIN DE GRADO

INGENIERÍA INFORMÁTICA

Diseño e implementación de un tutor inteligente basado en IA generativa y técnicas RAG para el aprendizaje guiado de programación

Autor

Luis Arrabal Gutiérrez

Directores

Kawtar Benghazi Akhlaki Sekkate



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

—

Granada, Junio de 2026

Diseño e implementación de un tutor inteligente basado en IA generativa y técnicas RAG para el aprendizaje guiado de programación

Autor
Luis Arrabal Gutiérrez

Directores
Kawtar Benghazi Akhlaki Sekkate

Diseño e implementación de un tutor inteligente basado en IA generativa y técnicas RAG para el aprendizaje guiado de programación

Luis Arrabal Gutiérrez

Palabras clave: RAG, LLM, MCP, Inteligencia Artificial, Educación Socrática.

En los últimos años, la rápida evolución de la Inteligencia Artificial Generativa ha transformado radicalmente el paradigma educativo. La llegada de los Grandes Modelos de Lenguaje (LLMs) presenta un impacto en la enseñanza de la programación y el desarrollo de software [1]. Aunque estas herramientas ofrecen capacidades avanzadas de resolución de problemas, plantean un desafío crítico: actúan proporcionando soluciones directas y código funcional, reduciendo el desarrollo del pensamiento del alumno, y sufren de alucinaciones al generar respuestas desconectadas del temario oficial de las asignaturas.

Para dar respuesta a este problema, este Trabajo de Fin de Grado propone y desarrolla *TutorIA*, una plataforma educativa avanzada basada en una arquitectura multi-agente inteligente. El sistema está especializado en las asignaturas de Fundamentos de la Programación (FP), Metodología de la Programación (MP) y Fundamentos del Software (FS) del Grado en Ingeniería Informática de la Universidad de Granada. *TutorIA* integra un entorno de desarrollo interactivo (IDE) en la nube, con editor de código y terminal virtual, orquestado por un tutor socrático que guía al estudiante mediante la formulación de preguntas socráticas en lugar de proporcionar respuestas directas.

A nivel arquitectónico, el sistema reduce las alucinaciones del modelo mediante la implementación de un motor de Generación Aumentada por Recuperación (RAG) conectado a una base de datos vectorial (Supabase), asegurando que las explicaciones se fundamenten estrictamente en los apuntes oficiales de la universidad, proporcionados por los profesores, y completando con información extra de internet. Además, introduce un sistema de perfilado que analiza en segundo plano las fortalezas y carencias del alumno, adaptando dinámicamente el diálogo y las preguntas.

El proyecto ha sido construido sobre un stack tecnológico moderno, seguro y escalable, utilizando React y TypeScript en el frontend, y Python (FastAPI y LangGraph) en el backend. La seguridad y el aislamiento de la ejecución de código (*sandboxing*) se garantizan mediante contenedores Docker efímeros, comunicados en tiempo real a través de WebSockets, mientras que el despliegue continuo (CI/CD) se apoya en infraestructuras cloud (Google Cloud, Vercel y PM2).

Finalmente, se ha llevado a cabo una fase exhaustiva de pruebas y validación del sistema evaluando casos de uso reales, lo que ha permitido confirmar

la estabilidad técnica de la plataforma y su viabilidad pedagógica como herramienta de apoyo al estudio.

Design and Implementation of an Intelligent Tutor Based on Generative AI and RAG Techniques for Guided Programming Learning

Luis Arrabal Gutiérrez

Keywords: RAG, LLM, MCP, Artificial Intelligence, Socratic Education

Abstract

In recent years, the fast evolution of Generative Artificial Intelligence has radically transformed the educational paradigm. The advent of Large Language Models (LLMs) has significantly impacted the teaching of programming and software development [1]. Although these tools offer advanced problem-solving capabilities, they present a critical challenge: providing direct solutions and functional code, they hinder the development of students' critical thinking. Furthermore, they are prone to hallucinations, generating responses that are disconnected from the official course syllabus.

To address this issue, this Bachelor's Thesis proposes and develops *TutorIA*, an advanced educational platform based on an intelligent multi-agent architecture. The system is specialized in the courses Fundamentals of Programming (FP), Programming Methodology (MP), and Software Fundamentals (FS) within the Computer Engineering Degree at the University of Granada. *TutorIA* integrates a cloud-based interactive development environment (IDE) with a code editor and a virtual terminal, orchestrated by a Socratic tutor that guides the student through targeted questions rather than providing direct answers.

At the architectural level, the system mitigates model hallucinations by implementing a Retrieval-Augmented Generation (RAG) engine connected to a vector database (Supabase). This ensures that explanations are strictly grounded in the official lecture notes provided by the faculty, supplemented with additional web resources. Additionally, it introduces a background profiling system that analyzes the student's strengths and weaknesses to dynamically adapt the dialogue and the generated questions.

The project is built upon a modern, secure, and scalable technology stack, using React and TypeScript for the frontend, and Python (FastAPI and LangGraph) for the backend. Security and code execution isolation (*sandboxing*) are guaranteed through ephemeral Docker containers communicating in real-time via WebSockets, while continuous deployment (CI/CD) is supported by cloud infrastructures (Google Cloud, Vercel, and PM2).

Finally, a comprehensive testing and validation phase of the system was conducted evaluating real use cases, confirming the technical stability of the platform and its pedagogical viability as a study support tool.

Yo, **Luis Arrabal Gutiérrez**, alumno de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 77166726R, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Luis Arrabal Gutiérrez

Granada a 22 de Junio de 2026 .

Dña. **Kawtar Benghazi Akhlaki Sekkate**, Profesora del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***Diseño e implementación de un tutor inteligente basado en IA generativa y técnicas RAG para el aprendizaje guiado de programación***, ha sido realizado bajo su supervisión por **Luis Arrabal Gutiérrez**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 22 de Junio de 2026 .

Los directores:

Kawtar Benghazi Akhlaki Sekkate

Agradecimientos

En primer lugar, deseo expresar mi más sincero agradecimiento a mi tutora, Dña. Kawtar Benghazi Akhlaki Sekkate, por su excelente y dedicada labor de dirección a lo largo de todos estos meses. Su rigor académico, su criterio técnico y su constante disposición para orientarme han resultado pilares imprescindibles para la correcta fundamentación teórica y práctica de este trabajo. Gracias a su supervisión y a sus valiosos consejos, ha sido posible abordar la resolución de los complejos retos tecnológicos que ha planteado el desarrollo de este proyecto, guiándome en todo momento con enorme profesionalidad.

También, hago este agradecimiento al profesor de la asignatura de Metodología de la Programación, D. Daniel Peralta Cámara. La cesión desde el inicio del material docente oficial ha constituido la base de conocimiento principal del sistema RAG, indispensable para garantizar su precisión y utilidad en un entorno universitario real.

Finalmente, mi profunda gratitud hacia mi familia y allegados. Su incondicional apoyo y comprensión durante todos estos años de estudio en la Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada han sido el soporte fundamental para la finalización de esta etapa formativa.

Índice general

1. Introducción **1**
- 1.1. Motivación y contexto . . . 1
- 1.2. El reto de la IA en la educación . . . 1
  - 1.2.1. Resolución automática de ejercicios a través de LLM . 2
  - 1.2.2. Generación de alucinaciones . . . 2
- 1.3. Objetivos del Proyecto (General y Específicos) . . . 3
  - 1.3.1. Objetivo general . . . 3
  - 1.3.2. Objetivos específicos . . . 3

2. Estado del Arte **4**
- 2.1. Introducción . . . 4
- 2.2. Sistemas de Tutoría Inteligente . . . 4
- 2.3. Fundamentos Tecnológicos . . . 5
  - 2.3.1. LLM y Prompt Engineering . . . 5
  - 2.3.2. Generación Aumentada por Recuperación (RAG) . . . 8
  - 2.3.3. Agentes Inteligentes . . . 12
  - 2.3.4. Protocolo MCP (Model Context Protocol) . . . 15
- 2.4. Análisis de soluciones existentes . . . 17
  - 2.4.1. Modelos de propósito general . . . 17
  - 2.4.2. Asistentes de codificación y productividad . . . 18
  - 2.4.3. Sistemas de tutoría especializada (Sakshm AI, CS50 Duck) . . . 19
  - 2.4.4. Comparación con este TFG . . . 20

3. Metodología y Gestión del Proyecto **22**
- 3.1. Introducción . . . 22
- 3.2. Metodología de Desarrollo . . . 22
- 3.3. Análisis de Requisitos . . . 23
  - 3.3.1. Requisitos Funcionales . . . 23
  - 3.3.2. Requisitos No Funcionales . . . 24
- 3.4. Planificación Temporal . . . 24
- 3.5. Presupuesto . . . 25
  - 3.5.1. Estimación del coste de desarrollo . . . 25
  - 3.5.2. Consideraciones Éticas y Privacidad . . . 26

4. Diseño e Implementación del Sistema **28**
- 4.1. Introducción . . . 28
- 4.2. Visión General de la Arquitectura . . . 28
  - 4.2.1. Stack Tecnológico y Justificación . . . 30

4.2.2. Ciclo de vida de una interacción 31
4.3. Capa de Presentación: Interfaz del Usuario (UI) 32
4.4. Capa Lógica y Ecosistema Multi-Agente 33
4.4.1. El Agente Principal: Tutor Socrático 34
4.4.2. El Agente Secundario: Analista Asíncrono 35
4.4.3. Ecosistema de Herramientas MCP 36
4.5. Integración de Ejecución Remota (Docker) 38
4.5.1. Aislamiento de código con Docker 38
4.5.2. Comunicación bidireccional mediante WebSockets . . 40
4.6. Modelo de Datos y Motor RAG (PostgreSQL y Supabase) . . 41
4.6.1. Manejo de Sesiones Stateless y Políticas RLS 41
4.6.2. Procesamiento y vectorización de apuntes (RAG) . . . 42
4.7. Despliegue Continuo 45
4.7.1. Capa de Presentación: *Edge Network* 45
4.7.2. Infraestructura del Backend (IaaS) 45
4.7.3. Automatización del Despliegue (CI/CD) 45
4.7.4. Gestión de Procesos y Enrutamiento Seguro 47

5. Pruebas y Validación del Sistema 48
5.1. Introducción 48
5.2. Validación de la Infraestructura de Ejecución y Aislamiento . 48
5.2.1. Aislamiento de Red y Restricciones de Hardware . . . 48
5.2.2. Estabilidad del Despliegue en la Nube 49
5.3. Pruebas de Integración en el Motor de Ejecución 50
5.3.1. Compilación Multi-Archivo 50
5.3.2. Detección de Bucles Infinitos y Limpieza de Recursos 50
5.4. Seguridad Perimetral, Memoria y Control de Abusos 51
5.4.1. Prevención de Spam y Limitación de Peticiones (Rate Limiting) 52
5.4.2. Control de Memoria RAM en la Subida de Archivos . 52
5.5. Evaluación del Subsistema de Inteligencia Artificial 53
5.5.1. Detección de Errores Lógicos en Proyectos Reales . . . 53
5.5.2. Contexto de la Terminal y Peticiones Canceladas . . . 54
5.6. Pruebas con Usuarios Reales (Beta Testing) 55
5.6.1. Metodología de la Evaluación 55
5.6.2. Resultados y Experiencia de Usuario 55
5.7. Análisis de Casos: Éxitos y Alucinaciones 56
5.7.1. Casos de Éxito: Recuperación Precisa y Ejecución Correcta 57
5.7.2. Manejo de Errores 58
5.7.3. Alucinaciones y Errores de Formato 59
5.7.4. Mejoras Detectadas para el Futuro 60

6. Conclusiones y Trabajos Futuros 62

6.1. Conclusiones 62
6.2. Limitaciones y Desafíos Técnicos 63
6.3. Dificultades Encontradas 64
6.4. Trabajos Futuros 65

Bibliografía **71**

Capítulo 1

Introducción

1.1. Motivación y contexto

En determinadas áreas tecnológicas la creciente masificación de las aulas, y la rápida evolución del conocimiento que dificulta la actualización continua de los contenidos impartidos exclusivamente mediante metodologías tradicionales, provocan que muchos estudiantes recurran cada vez más al estudio autónomo fuera del aula. En este contexto, la aparición de herramientas basadas en Inteligencia Artificial, especialmente los grandes modelos de lenguaje (LLM por sus siglas en inglés), ha facilitado el acceso rápido a explicaciones y recursos educativos en cualquier momento. Como consecuencia, numerosos alumnos utilizan estas herramientas como apoyo complementario al aprendizaje tradicional. [2][3]

Sin embargo, el uso extendido de estos sistemas introduce nuevos desafíos. Por un lado, existe el riesgo de una dependencia excesiva de respuestas generadas automáticamente, lo que puede reducir el desarrollo del razonamiento crítico del estudiante. Por otro, los modelos generalistas pueden presentar limitaciones como las denominadas "alucinaciones" o la generación de respuestas que, aunque aceptables, no son del todo correctas o adecuadas al contexto académico específico [4]. Además, estos sistemas no están necesariamente alineados con los criterios de evaluación ni con la metodología docente de cada asignatura, lo que puede generar soluciones correctas desde un punto de vista técnico-teórico, pero insuficientes desde una perspectiva específica para la asignatura. En este sentido, se hace necesario el desarrollo de herramientas que no solo proporcionen respuestas, sino que estén orientadas a fomentar el aprendizaje guiado y especializado en las diferentes asignaturas que se imparten.

1.2. El reto de la IA en la educación

Esta sección presenta 3 puntos que reflejan problemas de los LLM en el ámbito de la educación.

1.2.1. Resolución automática de ejercicios a través de LLM

Es notable que, durante los últimos años, el uso de LLM se ha expandido en múltiples ámbitos puesto que nos permite tener información casi instantánea. Sin embargo, uno de los ámbitos donde más destaca y más repercusión está teniendo es el ámbito académico, y esto nos lleva a preguntarnos cómo afecta el uso prolongado de los LLM en nuestra educación. Pues, según el estudio hecho en [5] encontraron que la integración de los LLM en los alumnos que estudiaban programación les ayudó a mejorar su pensamiento lógico en relación con la programación, aunque el estudio hecho en [6] se encontró que la mejora que producía el uso de los LLM era solo superficial para algunos estudiantes.[7]

1.2.2. Generación de alucinaciones

Una alucinación es una respuesta inconsistente o incorrecta, es decir, una respuesta que está desviada de la que busca el usuario. En la programación puede darse en contextos donde un alumno pide resolver un trozo de código, y que este LLM le genere una alucinación invisible, dando como resultado que el código compile, no dando error de primeras, pero que no esté bien ya sea porque no gestiona la memoria adecuadamente, genera código erróneo constantemente o por diversos paradigmas que se dan en el contexto de la programación, como los que se encuentran en el trabajo de investigacion realizado en [8], generando una sensación de desconfianza en estas herramientas. Además, estos modelos tampoco tienen en cuenta factores como el nivel de conocimiento del alumno, la metodología seguida por el profesorado o los criterios específicos de evaluación para cada práctica o ejercicio. Como consecuencia, el estudiante puede obtener una solución aparentemente correcta desde un punto de vista técnico, pero que no contribuya realmente al proceso de aprendizaje esperado.

Por ello, surge la necesidad de desarrollar herramientas educativas capaces de adaptarse al contexto y contenidos de una asignatura concreta, guiando al estudiante durante el proceso de resolución de ejercicios en lugar de limitarse a proporcionar respuestas directas. En este contexto, el proyecto propone el desarrollo de una plataforma de tutoría inteligente basada en técnicas RAG, capaz de utilizar apuntes y material docente oficial para ofrecer respuestas más alineadas con la asignatura y fomentar un aprendizaje guiado mediante interacción socrática.

1.3. Objetivos del Proyecto (General y Específicos)

1.3.1. Objetivo general

El objetivo principal de este Trabajo de Fin de Grado es diseñar e implementar una plataforma web de tutoría inteligente basada en Inteligencia Artificial generativa, orientada al aprendizaje guiado de programación mediante preguntas socráticas, garantizando la contextualización del conocimiento, la seguridad en la ejecución de código y la adaptación al nivel del estudiante.

1.3.2. Objetivos específicos

- Diseñar e implementar una plataforma web interactiva que permita la comunicación entre el estudiante y un sistema de tutoría basado en modelos de inteligencia artificial.
- Desarrollar un sistema de tutoría socrática basado en LLM, cuyo objetivo sea guiar el proceso de razonamiento del estudiante mediante preguntas en lugar de proporcionar soluciones directas.
- Integrar un sistema de Generación Aumentada por Recuperación (RAG) que permita contextualizar las respuestas del modelo utilizando material docente oficial de la asignatura.
- Implementar un entorno de ejecución seguro mediante contenedores aislados, que permita la validación y prueba de código sin comprometer el sistema.
- Diseñar un sistema de perfilado del estudiante que permita adaptar las respuestas del sistema en función de su nivel de conocimiento y progreso.
- Reducir la aparición de respuestas incorrectas o no contextualizadas mediante el uso de información verificada procedente del material docente.
- Realizar pruebas técnicas y validar el funcionamiento del sistema mediante casos de uso reales, garantizando la eficacia pedagógica del tutor y la estabilidad de la arquitectura propuesta.

El código fuente íntegro desarrollado para la implementación de este sistema, así como la documentación técnica asociada, se encuentran a disposición pública en el siguiente repositorio: `https://github.com/LuisArrabalGutierrez/tutor-ai/`.

Capítulo 2

Estado del Arte

2.1. Introducción

En este capítulo vamos a presentar los fundamentos teóricos y tecnológicos relacionados con los sistemas de tutoría inteligente como *TutorIA*, así como un análisis comparativo de soluciones existentes en el ámbito educativo y de asistencia a la programación.

2.2. Sistemas de Tutoría Inteligente

Los Sistemas de Tutoría Inteligente (Intelligent Tutoring Systems, ITS) son sistemas informáticos diseñados para apoyar el aprendizaje de forma personalizada, simulando determinadas funciones de un tutor humano mediante técnicas de inteligencia artificial. A lo largo de su evolución, estos sistemas han experimentado una transformación significativa. Las primeras generaciones se basaban en reglas predefinidas y árboles de decisión, lo que limitaba su capacidad para gestionar situaciones no contempladas durante el diseño del sistema. En la actualidad, los avances en inteligencia artificial y, especialmente la aparición de los modelos de lenguaje de gran escala (LLM), permiten a los ITS generar respuestas dinámicas, adaptarse al contexto del estudiante y ofrecer experiencias de aprendizaje más flexibles y personalizadas.

Desde el punto de vista pedagógico, los ITS pueden implementar diferentes estrategias de enseñanza con el objetivo de adaptarse a las necesidades y características de cada estudiante. Entre las más utilizadas se encuentran la instrucción directa, el aprendizaje basado en problemas, las técnicas de andamiaje (scaffolding) y el método socrático. Cada una de estas aproximaciones presenta ventajas específicas dependiendo del dominio de aplicación, el perfil del alumnado y los objetivos de aprendizaje perseguidos. En este trabajo se presta especial atención al método socrático debido a su relevancia en entornos educativos orientados al desarrollo del razonamiento y la resolución de problemas.

Método Socrático

El método socrático es una estrategia de enseñanza basada en el diálogo cuyo objetivo no es transmitir la información de manera directa, sino guiar al estudiante mediante una secuencia de preguntas para que alcance por sí mismo la comprensión de un concepto o la resolución de un problema. Este enfoque fomenta la reflexión, el pensamiento crítico y la construcción activa del conocimiento, en contraste con modelos de enseñanza basados exclusivamente en la exposición de contenidos.

En el ámbito de la programación este método resulta especialmente útil, ya que proporcionar una solución completa de forma inmediata puede reducir las oportunidades de aprendizaje. Por el contrario, guiar al estudiante mediante preguntas, pistas o sugerencias favorece el desarrollo de la lógica de programación, el pensamiento computacional y las capacidades de resolución de problemas, competencias fundamentales para la formación de programadores.

2.3. Fundamentos Tecnológicos

2.3.1. LLM y Prompt Engineering

Un Gran Modelo de Lenguaje o LLM (Large Language Model) es un sistema de inteligencia artificial diseñado para procesar, comprender y generar lenguaje natural. Estos modelos se basan principalmente en arquitecturas de aprendizaje profundo (*Deep Learning*), siendo los modelos *Transformer* la tecnología predominante en los sistemas más avanzados de la actualidad [4].

Los *Transformer* están construidos a partir de redes neuronales artificiales (RNA), modelos computacionales inspirados en el funcionamiento del cerebro humano que son capaces de aprender patrones complejos a partir de grandes volúmenes de datos. Las redes neuronales constituyen la base del aprendizaje profundo y representan uno de los pilares fundamentales sobre los que se sustentan los sistemas modernos de inteligencia artificial.

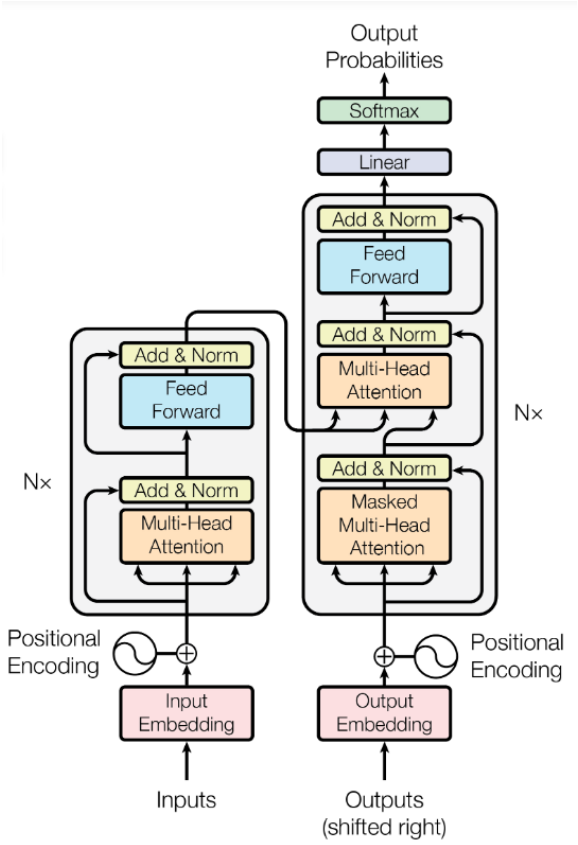


Figura 2.1: Modelo de Arquitectura Transformer

Arquitectónicamente una RNA está compuesta por unidades de procesamiento discretas llamadas nodos, las cuales se organizan de forma jerárquica en diferentes capas:

- **Capa de entrada (*Input layer*):** Recibe los datos iniciales.
- **Capas ocultas (*Hidden layers*):** Capas intermedias donde se realiza el procesamiento matemático complejo.
- **Capa de salida (*Output layer*):** Emite la predicción o el resultado final.

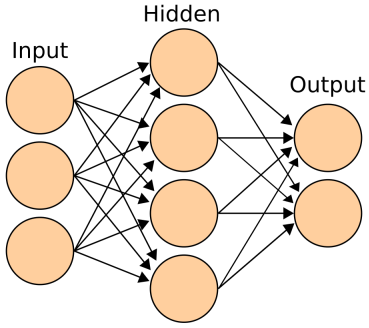


Figura 2.2: Estructura básica de una Red Neuronal

Para que un modelo o LLM alcance un rendimiento óptimo capaz de asistir en tareas complejas como la programación, su ciclo de vida se divide en 3 fases de entrenamiento, cada una con un objetivo distinto [9]:

- **Pre-entrenamiento supervisado (*Pre-training*):** En esta fase inicial, el modelo de inteligencia artificial procesa volúmenes masivos de texto. El objetivo es puramente probabilístico, ya que se basa en predecir el siguiente fragmento de palabra (*token*) de una secuencia. En esta fase, la red neuronal ajusta sus miles de millones de parámetros para asimilar la sintaxis, la semántica, el conocimiento general, etc. Por tanto, al terminar esta fase, el modelo pre-entrenado solo sabe "completar" texto.
- **Ajuste Fino Supervisado (*Supervised Fine-Tuning*, SFT):** Para mejorar el modelo base a un modelo útil, se somete a un segundo entrenamiento utilizando un segundo conjunto de datos más reducido, pero de mayor calidad. Este conjunto suele estar compuesto por pares de "instrucción-respuesta" verificados por expertos. Cuando esta etapa termina, el modelo ha modificado sus conocimientos para aprender el formato de preguntas y respuestas, adquiriendo la capacidad de obedecer directrices de los usuarios.
- **Aprendizaje por Refuerzo con Retroalimentación Humana (*Reinforcement Learning from Human Feedback*, RLHF):** En esta etapa se entrena un "modelo de recompensa" paralelo, para alinear el comportamiento con los valores humanos y la seguridad. Varios evaluadores humanos califican las respuestas generadas por el LLM, premiando las que son útiles, precisas e inofensivas. Por último, se utilizan algoritmos para ajustar el LLM principal, maximizando la probabilidad de generar respuestas que tengan una alta puntuación.

Tras este "entrenamiento" [9, 4], el LLM sigue siendo probabilístico, es decir, la respuesta y su comportamiento depende en gran manera del

contexto que se le proporcione. Por eso, para controlar su comportamiento se emplea el *Prompt Engineering* (Ingeniería de Instrucciones).

El *Prompt Engineering* consiste en el diseño, estructuración y optimización de los textos de entrada (*prompt*) para guiar al LLM hacia la salida esperada sin necesidad de modificar o reentrenar sus parámetros [7].

Dentro del *Prompt Engineering*, la configuración del *System Prompt* (Instrucción de Sistema) resulta imprescindible. Se trata de una instrucción invisible para el usuario final que define la "personalidad" de la IA, el contexto, las restricciones y las reglas a nivel de sistema. En el caso de *TutorIA*, el *System Prompt* es el mecanismo de seguridad que obliga al modelo a adoptar un rol socrático, prohibiéndole explícitamente entregar código fuente resuelto y forzándole a auditar el progreso del alumno mediante preguntas estratégicas.

Además, la ingeniería de instrucciones en ámbitos lógicos y de programación se apoya en técnicas avanzadas como la "Cadena de Pensamiento" (*Chain-of-Thought* o CoT). Esta técnica induce al modelo a desglosar y razonar el problema de forma secuencial, obligándole a generar los pasos intermedios de su deducción antes de mandar una respuesta final. El uso de CoT reduce significativamente las alucinaciones del modelo y mejora de forma drástica su capacidad para guiar al estudiante de manera estructurada y coherente [10].

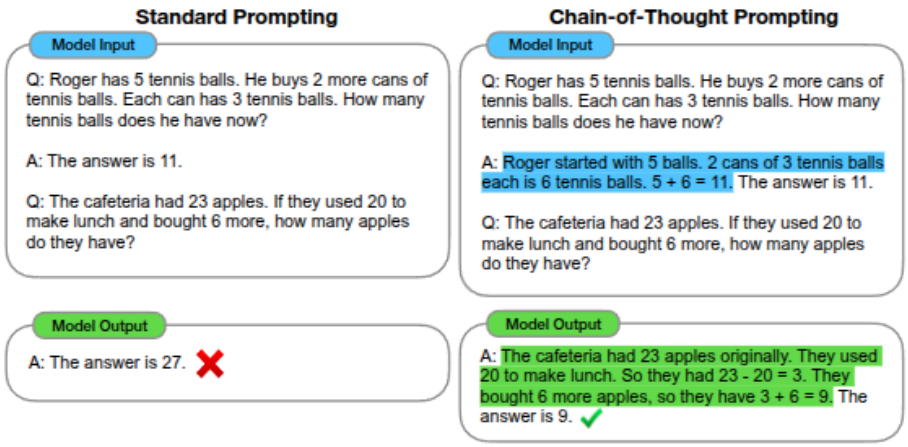


Figura 2.3: Comparación entre el Prompt Estándar (izquierda) y el Chain of Thought (derecha) [10].

2.3.2. Generación Aumentada por Recuperación (RAG)

El problema que presentan los LLM es la tendencia a alucinar en determinados aspectos, puesto que carecen de conocimiento específico sobre

normativas, pautas o temarios particulares de una universidad o asignatura. Para solucionar este problema se presentó la técnica RAG (*Retrieval Augmented Generation*), que consiste en la inyección de contexto específico de documentos directamente al LLM, ayudando a que el LLM tenga el contexto adecuado y mitigando las alucinaciones [11].

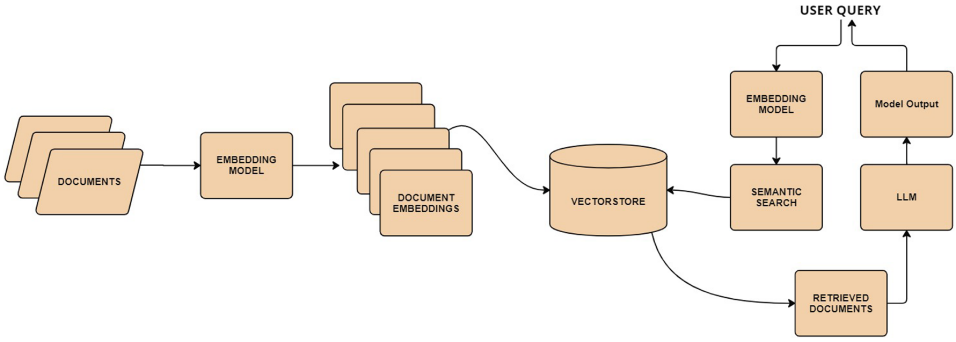


Figura 2.4: Arquitectura general de un sistema RAG [11].

Para que un sistema RAG funcione correctamente, se debe establecer un flujo de trabajo o *pipeline*, compuesto por las fases de indexación, recuperación y generación. Dentro del *Vector RAG*, que es el enfoque estándar, el primer paso es procesar los documentos originales, dividiéndose este proceso en dos etapas:

- **Limpieza de texto:** Es fundamental "limpiar" los datos eliminando caracteres invisibles, ruido de formato como etiquetas HTML, metadatos innecesarios, etc. El tener un texto limpio nos asegura que el modelo no asigne importancia a elementos que carecen de valor, mitigando así también posibles futuras alucinaciones [12].
- **Fragmentación (*Chunking*):** Dado que los documentos largos exceden la ventana de contexto (*tokens*) del LLM, el texto se divide en fragmentos o *chunks* más pequeños. Una fragmentación inteligente se suele configurar con un solapamiento de palabras entre bloques consecutivos para garantizar que no se pierda el hilo lógico ni el contexto entre el texto que ha sido dividido, por ejemplo, un fragmento podría ser equivalente a una diapositiva [12].

Una vez que han sido preparados y fragmentados los textos, se emplea el ***embedding***, siendo una representación matemática por la cual la información oficial (diapositivas del temario) y la pregunta del alumno se transforman en vectores dentro de un espacio vectorial. Al transformar estos conceptos, la semántica se representa geométricamente, así se consigue que dos palabras con un campo semántico parecido aparezcan en regiones cercanas,

ayudándonos a recuperar posteriormente la información por su significado o intención, y no por simples coincidencias de palabras exactas. Los vectores resultantes se almacenan en bases de datos vectoriales [12].

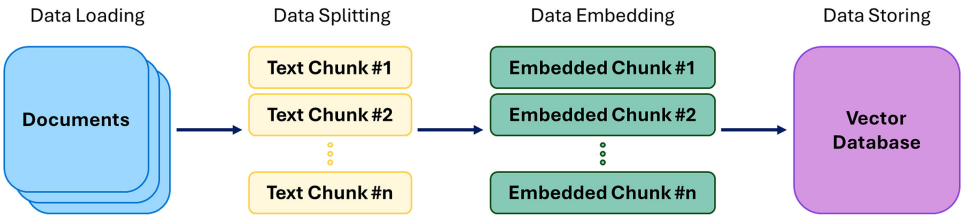


Figura 2.5: Proceso de fragmentación de la información y *embedding* [12].

La recuperación del contexto más preciso se realiza usando la similitud del coseno entre todos los vectores de la base de datos y la consulta de la base de datos. La similitud del coseno consiste en medir el ángulo que forman los 2 vectores a evaluar: la consulta del usuario, que ha sido transformada a vector mediante el *embedding*, y cada uno de los vectores de la base de datos. Al obtener el coseno entre ambos vectores, podemos saber la dirección de los mismos, siendo un valor más cercano a 1 el indicador de mayor similitud entre ambos vectores.

A continuación se presenta la fórmula para calcular el ángulo coseno entre dos vectores A y B:

$$\text{similitud}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}} \tag{2.1}$$

En esta ecuación, el numerador representa el producto escalar de los vectores, el cual evalúa qué tanto se alinean sus componentes, y el denominador actúa como un factor de normalización. Este enfoque presenta mejoras respecto a otras métricas como la Distancia Manhattan o la Distancia Euclídea, ya que nos importa la dirección de los vectores, que representa el significado semántico, y no la magnitud como en la Distancia Euclídea, que representa la longitud del texto [13].

Además, a medida que esta tecnología ha ido madurando, la arquitectura de *Vector RAG* ha evolucionado a otros sistemas más sofisticados para resolver problemas de recuperación complejos. Las arquitecturas modernas se expanden en varias direcciones para superar las limitaciones de la búsqueda vectorial:

- **RAG Híbrido (*Hybrid RAG*):** Este enfoque fusiona la búsqueda

vectorial (*embeddings* semánticos) con la búsqueda tradicional (algoritmos basados en frecuencia de palabras). En un contexto educativo de programación, esto resulta vital ya que permite al sistema entender la intención general de la duda del alumno mediante la semántica, sin perder la capacidad de encontrar coincidencias exactas de nombres de variables, funciones de código o errores de compilación específicos [12].

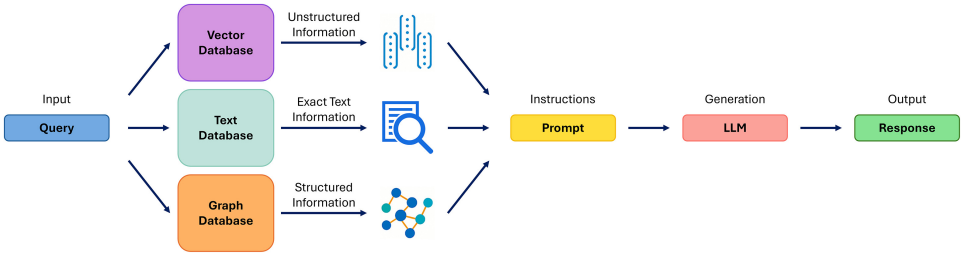


Figura 2.6: Arquitectura de un sistema de RAG Híbrido [12].

- ***Graph* RAG (Grafos de Conocimiento):** Consiste en integrar *Knowledge Graphs* dentro del *pipeline* de recuperación. En lugar de buscar fragmentos de texto aislados, la información del temario se estructura relacionando entidades y conceptos clave de forma jerárquica. Esto dota al modelo de la capacidad de realizar razonamientos de "múltiples saltos"(*multi-hop reasoning*), conectando conceptos separados en diferentes temas o documentos para generar respuestas mucho más completas y estructuradas [12].

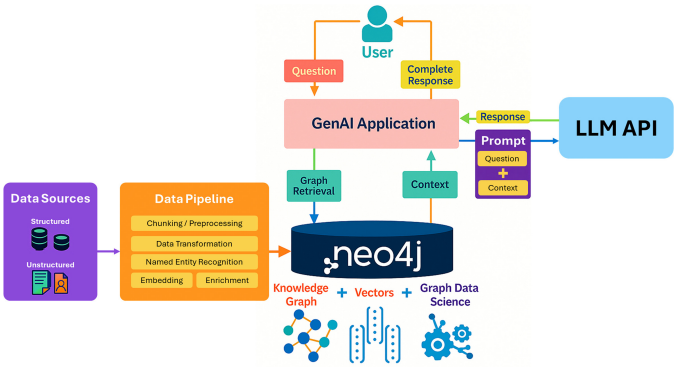


Figura 2.7: Flujo de un Graph RAG [12].

- **RAG Multimodal:** Expande la arquitectura para procesar simultáneamente diferentes tipos de datos, como texto e imágenes, añadiéndolos en un mismo espacio vectorial compartido. Para el alumno, esto se traduce en que la IA puede recuperar y analizar de forma conjunta el

texto y los diagramas de flujo, esquemas o capturas integradas en las diapositivas de la asignatura [12].

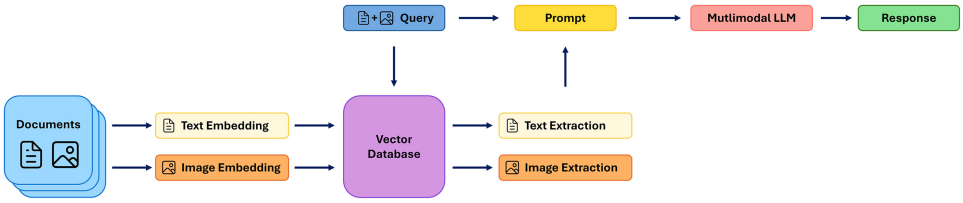


Figura 2.8: Flujo de un RAG Multimodal [12].

2.3.3. Agentes Inteligentes

Aunque los modelos de lenguaje de gran escala son capaces de generar texto, responder preguntas y resolver tareas complejas, por sí mismos constituyen sistemas reactivos: reciben una entrada y producen una salida. Un agente inteligente amplía estas capacidades incorporando mecanismos de planificación, memoria, acceso a herramientas externas y capacidad de actuación sobre un entorno [14].

De forma general, un agente puede definirse como una entidad software capaz de percibir información de su entorno, razonar sobre ella y ejecutar acciones orientadas a la consecución de un objetivo determinado. En el contexto de los LLM, un agente combina las capacidades de razonamiento y generación de lenguaje del modelo con herramientas externas que le permiten acceder a información actualizada, ejecutar código, consultar bases de datos o interactuar con otros sistemas.

Esta evolución representa un cambio de paradigma respecto al uso tradicional de los modelos de lenguaje. Mientras que un LLM aislado se limita a responder una consulta, un agente puede descomponer problemas complejos en subtareas, planificar una secuencia de acciones y adaptar su comportamiento en función de los resultados obtenidos durante la ejecución [10].

Arquitecturas Multi-Agente

A medida que aumenta la complejidad de las tareas, un único agente puede resultar insuficiente para gestionar de forma eficiente todas las responsabilidades del sistema. Como respuesta a esta limitación surgen las arquitecturas multi-agente, en las que múltiples agentes especializados colaboran para alcanzar un objetivo común [15].

En lugar de centralizar toda la lógica en un único componente, las arquitecturas modernas abordan la complejidad mediante una estricta división del

trabajo basada en el "juego de roles". Un claro referente de esta metodología es el framework CrewAI [16], diseñado específicamente para estructurar la colaboración entre agentes especialistas. Bajo este paradigma, cada agente asume una responsabilidad concreta y exclusiva; por ejemplo, un agente puede dedicarse a la interacción con el usuario, otro a la recuperación de información, un tercero a la ejecución de herramientas externas y un último a la evaluación de los resultados. Esta comunicación y coordinación permite resolver tareas que desbordarían a un sistema monolítico.

Las arquitecturas multi-agente presentan ventajas significativas en términos de modularidad, escalabilidad y mantenibilidad. La especialización de los agentes facilita la reutilización de componentes y permite modificar o sustituir determinadas funcionalidades sin afectar al conjunto del sistema. Además, este enfoque se alinea con los principios clásicos de ingeniería del software, especialmente con la separación de responsabilidades y el diseño modular.

En los últimos años, este paradigma ha ganado relevancia debido al auge de los LLM y de los sistemas basados en agentes, convirtiéndose en una de las aproximaciones más utilizadas para el desarrollo de asistentes inteligentes avanzados, sistemas autónomos y plataformas capaces de coordinar múltiples fuentes de información y herramientas externas [17].

Tool Calling

Uno de los principales retos de los modelos de lenguaje de gran escala es que su conocimiento está limitado a la información disponible durante su entrenamiento y a la ventana de contexto proporcionada en cada interacción. Como consecuencia, los LLM no pueden acceder directamente a información actualizada ni interactuar de forma nativa con sistemas externos. Para superar estas limitaciones surge el concepto de *Tool Calling* (Llamada a Herramientas).

El *Tool Calling* es un mecanismo mediante el cual un modelo de lenguaje puede solicitar la ejecución de herramientas externas cuando necesita obtener información adicional o realizar una acción específica. Estas herramientas pueden incluir consultas a bases de datos, ejecución de código, acceso a APIs, recuperación de documentos, lectura de ficheros o interacción con otros servicios *software*.

Para ello, las herramientas se describen mediante esquemas estructurados que especifican su nombre, funcionalidad y parámetros de entrada. Dichos esquemas son incorporados al contexto del modelo durante la inferencia. Cuando el LLM determina que una herramienta puede ayudarle a resolver una tarea, en lugar de generar una respuesta en lenguaje natural produce una estructura de datos, normalmente en formato JSON, que indica

qué herramienta debe utilizarse y qué parámetros deben proporcionarse.

La aplicación anfitriona interpreta esta solicitud, ejecuta la herramienta correspondiente y devuelve el resultado al modelo. A partir de esta nueva información, el LLM puede continuar su proceso de razonamiento y generar una respuesta más precisa y fundamentada. Este enfoque transforma al modelo de lenguaje en un componente capaz de interactuar con sistemas externos, ampliando significativamente sus capacidades más allá de la simple generación de texto[18].

La aparición de estándares como el *Model Context Protocol* (MCP) ha contribuido a formalizar este mecanismo, proporcionando una forma estandarizada de conectar modelos de lenguaje con herramientas, servicios y fuentes de datos externas, favoreciendo la interoperabilidad entre diferentes proveedores y aplicaciones [18].

Orquestación

A medida que aumenta el número de agentes, herramientas y fuentes de información disponibles, surge la necesidad de coordinar de forma eficiente todas las interacciones dentro del sistema. Este proceso de coordinación recibe el nombre de *orquestación*.

La orquestación puede entenderse como la capa encargada de gestionar el flujo de ejecución de un sistema basado en agentes. Su función principal consiste en decidir qué acciones deben realizarse, en qué orden deben ejecutarse y qué información debe intercambiarse entre los distintos componentes del sistema. En otras palabras, actúa como un mecanismo de control que coordina el comportamiento global de la aplicación [14].

En arquitecturas sencillas, la orquestación puede implementarse mediante flujos secuenciales predeterminados. Sin embargo, en sistemas complejos resulta necesario utilizar mecanismos más dinámicos que permitan adaptar el flujo de trabajo en función del contexto y de los resultados obtenidos durante la ejecución. Este enfoque es especialmente relevante en aplicaciones basadas en inteligencia artificial, donde el comportamiento del sistema puede variar significativamente en función de la consulta realizada por el usuario [19].

La orquestación también desempeña un papel fundamental en las arquitecturas multi-agente. En estos entornos, cada agente suele estar especializado en una tarea concreta y la capa de orquestación es responsable de coordinar la colaboración entre ellos, garantizando que la información fluya correctamente y que las tareas se ejecuten de forma coherente. Gracias a este enfoque, es posible construir sistemas modulares, escalables y fácilmente mantenibles.

En los últimos años han surgido diversos marcos de trabajo orientados a la construcción de sistemas basados en agentes y flujos orquestados, como LangGraph [20], AutoGen [15], CrewAI [16] o Semantic Kernel [21], que facilitan la definición de procesos complejos y la coordinación entre múltiples componentes inteligentes.

Paradigma ReAct

Uno de los enfoques más influyentes en el diseño de agentes basados en modelos de lenguaje es el paradigma *ReAct* (*Reasoning and Acting*), propuesto por Yao et al. en 2022 [19]. Este paradigma combina capacidades de razonamiento y ejecución de acciones dentro de un mismo ciclo de trabajo.

Tradicionalmente, un LLM recibe una consulta y genera directamente una respuesta. Sin embargo, muchas tareas complejas requieren obtener información adicional, realizar cálculos o consultar herramientas externas antes de poder emitir una respuesta adecuada. El paradigma ReAct aborda este problema mediante un proceso iterativo compuesto por tres fases principales: razonamiento, acción y observación.

En la fase de razonamiento (*Thought*), el modelo analiza el problema y determina qué información necesita para continuar. Posteriormente, en la fase de acción (*Action*), selecciona una herramienta o ejecuta una operación concreta. Finalmente, en la fase de observación (*Observation*), incorpora los resultados obtenidos a su contexto interno y utiliza esta nueva información para continuar razonando o generar una respuesta final.

Este ciclo puede repetirse múltiples veces hasta alcanzar una solución satisfactoria. Gracias a este enfoque, el modelo es capaz de descomponer problemas complejos en subtareas más simples, interactuar con herramientas externas y adaptar su comportamiento dinámicamente en función de los resultados obtenidos durante la ejecución.

La combinación de *Tool Calling*, orquestación y paradigma ReAct constituye actualmente uno de los pilares fundamentales de los sistemas basados en agentes. Estos mecanismos permiten superar muchas de las limitaciones inherentes a los modelos de lenguaje tradicionales y han impulsado el desarrollo de asistentes inteligentes, sistemas autónomos y plataformas educativas avanzadas capaces de interactuar activamente con su entorno.

2.3.4. Protocolo MCP (Model Context Protocol)

El protocolo MCP (*Model Context Protocol*) de Anthropic [22], introducido recientemente como estándar abierto en la industria, permite solucionar la fragmentación que se producía con las integraciones ad-hoc, mejorando la

escalabilidad y seguridad del *Tool Calling*. MCP permite conectar modelos inteligentes con herramientas locales o remotas de una manera segura, controlada y delimitada, resolviendo así el problema de interoperabilidad entre los LLM y otras fuentes de datos o entornos de ejecución.

MCP sigue una arquitectura cliente-servidor donde un *host* MCP (la aplicación de IA principal) establece conexiones con uno o más servidores MCP. Las primitivas fundamentales en la arquitectura MCP son [18]:

- **MCP *Host* (Modelo de IA):** Es la aplicación de IA que coordina y gestiona uno o varios clientes MCP.
- **Cliente MCP:** Mantiene la conexión (a través de canales como *stdio* o *SSE*) con el servidor MCP y obtiene el contexto de este para proporcionárselo al *Host* MCP.
- **Servidor MCP:** Programa ligero que proporciona contexto y capacidades de ejecución a los clientes MCP a través de tres pilares:
  - **Recursos (*Resources*):** Datos de solo lectura controlados por el servidor (como archivos locales o bases de datos) que el LLM puede leer para obtener contexto adicional.
  - **Plantillas (*Prompts*):** Estructuras de instrucciones reutilizables que guían el comportamiento del modelo para tareas específicas.
  - **Herramientas (*Tools*):** Funciones ejecutables que permiten al LLM realizar acciones que interactúan con APIs externas o el sistema operativo, operando siempre bajo la supervisión y los límites de seguridad impuestos por el servidor.

Gracias a este protocolo, el agente se transforma en un sistema capaz de procesar información de una manera reactiva en tiempo real y de realizar funcionalidades automáticamente, tales como compilar código o recuperar contexto externo. Además, investigaciones recientes destacan cómo el uso de servidores MCP facilita la colaboración entre múltiples agentes, optimizando flujos de trabajo complejos y reduciendo la fricción en el desarrollo [23].

Esta estandarización resulta fundamental para el entorno educativo propuesto en este proyecto, ya que permite aislar las bases de datos vectoriales (RAG) o los entornos de ejecución de código (*Sandboxing*) en servidores o procesos independientes, garantizando que el LLM pueda operar sobre ellos de forma autónoma sin comprometer en ningún momento la seguridad del sistema del usuario o del servidor anfitrión.

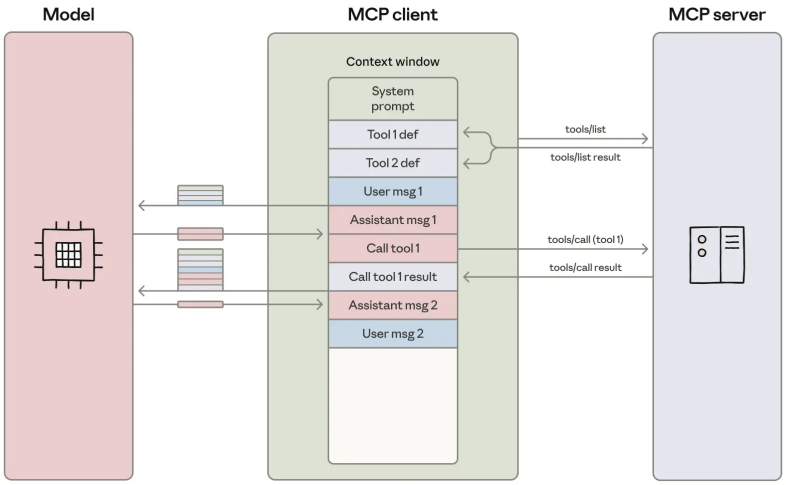


Figura 2.9: Arquitectura de MCP [22].

2.4. Análisis de soluciones existentes

La rápida evolución de la inteligencia artificial ha impulsado el desarrollo de múltiples herramientas de asistencia a la programación. Para entender su impacto real en la educación, es necesario analizar cómo se utilizan en la práctica. Si evaluamos sus fortalezas y carencias pedagógicas, podemos clasificar las soluciones actuales en tres grandes grupos.

2.4.1. Modelos de propósito general

Los modelos fundacionales de propósito general poseen un conocimiento masivo extraído de internet. Esto se logra mediante el procesamiento de volúmenes masivos de datos durante sus fases de entrenamiento, lo que les otorga un gran potencial para resolver problemas algorítmicos o de software complejo. Sin embargo, fallan de forma repetitiva en la pedagogía básica universitaria, ya que al enfrentarse a las dudas de un alumno, estas plataformas carecen de un enfoque didáctico y tienden a generar el código solucionado de manera directa y completa. Esta inmediatez en la respuesta altera significativamente el proceso de aprendizaje; en lugar de construir una solución paso a paso, el estudiante recibe el producto final, cambiando su rol de "creador" a "revisor". El problema radica en que, en etapas tempranas de formación educativa, el alumno aún no ha desarrollado el criterio necesario para revisar ese código. Como resultado, se asimilan estructuras y patrones de diseño sin comprender sus bases ni el control de flujo.

Estudios recientes demuestran que el uso de estas herramientas sin un diseño pedagógico impacta de manera negativa. Los alumnos que mantienen un contacto dependiente y repetitivo con estos asistentes terminan adquiriendo un conocimiento puramente superficial, donde la sintaxis se copia pero la lógica se desconoce.

Un ejemplo de esto es el estudio realizado en [6], donde se analizó el impacto de dejar a libre elección el uso de inteligencias artificiales como ChatGPT. Mediante pruebas de evaluación, se llegó a la conclusión de que los estudiantes que utilizaron el modelo con mayor frecuencia para generar soluciones tendieron a obtener una calificación más baja, en comparación con aquellos alumnos que lo emplearon únicamente como un complemento para entender conceptos teóricos.

2.4.2. Asistentes de codificación y productividad

En los últimos años, la integración de asistentes de programación con Inteligencia Artificial como GitHub Copilot, implementados directamente en el entorno de desarrollo, ha revolucionado la industria del desarrollo de software. Estas herramientas tienen como objetivo primario el aumento de la productividad para el desarrollo de software, automatizando tareas repetitivas o de autocompletado predictivo, es decir, intentan predecir según el contexto del archivo o del proyecto, las siguientes líneas de código. Estas herramientas han demostrado una gran capacidad de generar soluciones precisas y eficientes para determinados problemas [6].

Sin embargo, la calidad del código producido depende en gran medida de la claridad y especificidad de las instrucciones y contexto del proyecto proporcionado por el alumno o usuario [6]. Además, aunque las sugerencias parecen a simple vista correctas, la Inteligencia Artificial suele carecer del contexto global, lo que puede suponer la realización de implementaciones que no cumplen los requisitos deseados. Incluso, si la herramienta de Inteligencia Artificial ha sido entrenada con una enorme cantidad de código sin revisar, es seguro que el modelo habrá aprendido de código vulnerable o que contiene errores [24]. La capacidad de diferenciar si una solución es defectuosa requiere una experiencia técnica de la que la mayoría de los alumnos universitarios de primeros cursos carecen, siendo en este ámbito donde estas limitaciones tienen mayor repercusión, dándose una disminución de la efectividad de forma significativa cuando se usan con una carencia respecto al contexto del problema y técnicas de programación adecuada [6].

Especialmente, este problema resulta significativo en asignaturas introductorias de programación o de fundamentos de software, donde el objetivo principal no consiste en obtener un resultado correcto de forma rápida, sino en desarrollar habilidades de razonamiento lógico, algorítmico y de pen-

samiento computacional. En este contexto, el uso continuado de estas herramientas de Inteligencia Artificial basadas en el autocompletado puede provocar una dependencia de las sugerencias automáticas.

Además, investigaciones recientes sobre el impacto de modelos generativos en la enseñanza de programación sugieren que, aunque estas herramientas pueden acelerar determinadas tareas, también pueden fomentar un aprendizaje superficial cuando el estudiante se limita a reutilizar soluciones generadas automáticamente sin reflexionar sobre ellas o entender la lógica detrás de ellas [25].

Por tanto, aunque los asistentes de generación de código basados en IA representan un avance significativo en productividad y desarrollo de software, presentan importantes limitaciones desde una perspectiva pedagógica. Esto evidencia la necesidad de sistemas orientados específicamente al aprendizaje, capaces de guiar al estudiante durante el proceso de razonamiento y adaptación al contexto docente de cada asignatura, en lugar de centrarse únicamente en la generación automática de código.

2.4.3. Sistemas de tutoría especializada (Sakshm AI, CS50 Duck)

Como medida de reducción de las carencias pedagógicas que representan los modelos generalistas o los asistentes de productividad, han surgido iniciativas orientadas específicamente a la educación asistida por Inteligencia Artificial. Un gran referente es *CS50 Duck*, un asistente desarrollado por la Universidad de Harvard [26], y otras plataformas como *Sakshm AI*, enfocadas en estudiantes de programación de universidades en India [1]. A diferencia de otras herramientas, estas están diseñadas específicamente bajo directrices estrictas, haciendo que su comportamiento esté acotado para no facilitar código funcional y, en su lugar, guiar al estudiante mediante preguntas tal como plantea la enseñanza socrática, obligando al alumno a pensar y desarrollar de forma activa su propia lógica de programación.

Además, aunque no quepa discusión sobre su valor en el ámbito académico, estas herramientas presentan un inconveniente, puesto que están diseñadas para instituciones específicas o modelos de enseñanza particulares, dándose así una carencia de flexibilidad necesaria para adaptarse a otros temarios hiper-localizados, normativas o criterios específicos de otras universidades como la Universidad de Granada.

Asimismo, desde el punto de vista técnico, muchas de estas plataformas prescinden de funcionalidades que ayudan a tener una experiencia de tutorización completa. La mayoría operan como interfaces de conversación de texto y no disponen de entorno de ejecución interactivo seguro en la nube (*sandboxing*) que permitan a la IA auditar el código del alumno de forma

autónoma. Además, muchos carecen de sistemas de perfilado dinámico del alumno que sean capaces de evaluar y almacenar en segundo plano las fortalezas o debilidades del estudiante sin afectar el rendimiento y la experiencia de usuario.

2.4.4. Comparación con este TFG

A diferencia de las alternativas analizadas en la sección anterior, el valor principal de *TutorIA*, el proyecto desarrollado en este Trabajo Fin de Grado, no reside en competir por disponer del modelo de inteligencia artificial con un mayor número de parámetros o el mejor rendimiento en tareas de propósito general. Su principal aportación radica en la especialización y adaptación al contexto educativo de la Universidad de Granada, proporcionando funcionalidades específicamente diseñadas para las necesidades de estudiantes y docentes. Esta propuesta se sustenta en tres componentes fundamentales:

1. **Hiper-localización del contexto (RAG):** Las explicaciones teóricas y la resolución de dudas sobre programación (en C++) y en fundamentos de software (en Linux) se fundamentan de manera estricta en diapositivas y temario oficial proporcionado por los docentes de las asignaturas (FP, MP, FS). Esto garantiza una alineación total de las respuestas con el plan de estudios y evita a la IA la generación de "alucinaciones" y respuestas que no estén adecuadas a lo esperado.
2. **Seguridad y contexto unificado:** El proyecto integra un entorno de ejecución aislado mediante contenedores Docker alojados en una Máquina Virtual en la nube. Esto permite que la ejecución y compilación del alumno se haga de manera segura, teniendo un extra de seguridad a la hora de programar dado que, si algo falla, no afectaría al dispositivo del alumno, solo a un contenedor efímero que se reiniciaría. Además, se consigue que todos los alumnos ejecuten y compilen el código bajo el mismo Sistema Operativo (SO) y las mismas versiones de paquetes y librerías.
3. **Perfilado asíncrono:** Mediante un enfoque multi-agente, el agente principal interactúa con el alumno, y un agente secundario analiza la conversación en tiempo real en segundo plano. Este sistema extrae y actualiza tanto fortalezas como debilidades del estudiante, adaptándose dinámicamente al nivel del propio alumno para futuras interacciones, y tiene la gran ventaja de no ralentizar el tiempo de respuesta ni perjudicar la experiencia del usuario.

A modo de representar estas diferencias de diseño y funcionalidades, en las siguientes figuras se muestran estas diferencias con los principales asistentes que se han analizado previamente y el proyecto realizado.

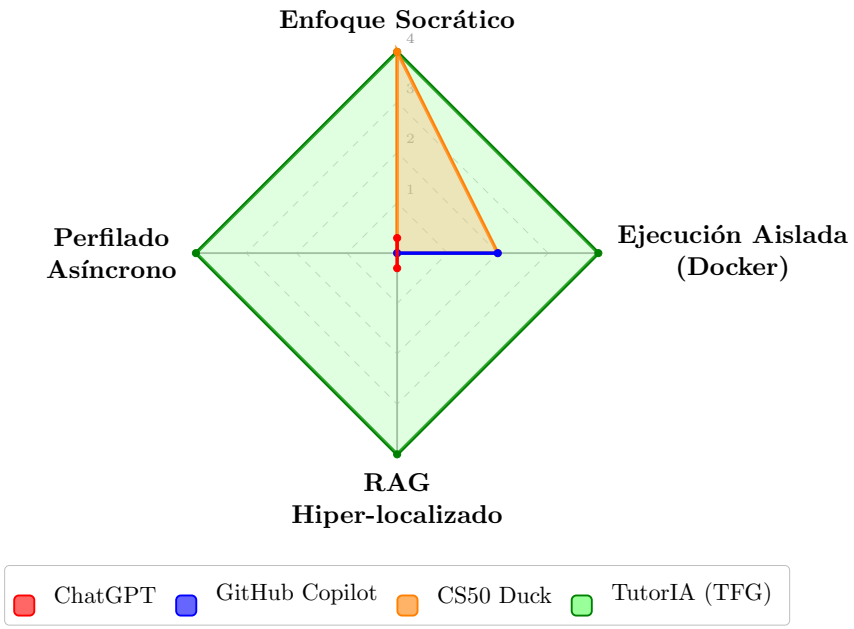


Figura 2.10: Comparativa funcional entre TutorIA y otras soluciones analizadas.

	Socrático	Docker	RAG UGR	Perfilado
ChatGPT	Mínimo	No	Mínimo	No
GitHub Copilot	No	Local / Parcial	No	No
CS50 Duck	Sí	Parcial	No	No
TutorIA (TFG)	Sí (Avanzado)	Sí (Nube)	Sí	Sí

Figura 2.11: Tabla comparativa de las características

Capítulo 3

Metodología y Gestión del Proyecto

3.1. Introducción

El desarrollo de una plataforma educativa interactiva como TutorIA requiere una planificación que garantice no solo la viabilidad técnica del sistema, sino también el cumplimiento de los plazos académicos y la correcta gestión de los recursos disponibles. En este capítulo se detalla el ciclo de vida del software, los requisitos técnicos extraídos de la fase de análisis, la planificación temporal de las iteraciones y, un estudio económico que estima el coste que habría supuesto el desarrollo de este proyecto en un entorno empresarial real.

3.2. Metodología de Desarrollo

Para la implementación de TutorIA, se ha optado por las metodologías ágiles, concretamente basada en Scrum. Dado que este Trabajo de Fin de Grado se desarrolla de forma individual, los roles tradicionales de Scrum (Scrum Master, Product Owner y Equipo de Desarrollo) los ejerce una única persona. Aún así, se han mantenido los principios fundamentales: desarrollo iterativo, entregas incrementales y adaptación continua ante los cambios.

El ciclo de vida del software se ha dividido en iteraciones o *sprints* de dos a tres semanas de duración. Al finalizar cada *sprint*, se obtenía un incremento funcional del producto (por ejemplo, el aislamiento en contenedores Docker o la integración de la API con el LLM).

Para el control de versiones y la gestión de tareas se ha empleado Git junto con la plataforma GitHub. El uso de tableros Kanban ha permitido clasificar los objetivos en tareas pendientes (*To Do*), en progreso (*In Progress*) y finalizadas (*Done*), facilitando la identificación de cuellos de botella durante las fases más críticas, como la configuración del sistema para el despliegue en la nube.

3.3. Análisis de Requisitos

La definición de los requisitos sobre los que se diseña la arquitectura del sistema es una de las partes más fundamentales del proyecto. A continuación, se clasifican en requisitos funcionales y no funcionales.

3.3.1. Requisitos Funcionales

Los requisitos funcionales describen las interacciones específicas y los servicios que el sistema debe proporcionar a los estudiantes y docentes:

- **RF-01. Gestión de usuarios y sesiones:** El sistema debe permitir la identificación única y autenticación de los alumnos mediante un token de sesión. Esto garantiza la gestión aislada de sus entornos de trabajo y la persistencia de su historial de interacciones.
- **RF-02. Entorno de edición de código:** La plataforma debe integrar un editor de código web con soporte específico para C++, incluyendo herramientas de apoyo a la programación como el resaltado de sintaxis y la numeración de líneas.
- **RF-03. Compilación y ejecución remota:** El servidor debe ser capaz de recibir los archivos del alumno, compilarlos usando herramientas estándar (`g++` o `make`) y ejecutarlos remotamente, devolviendo a la interfaz tanto la salida estándar generada como los posibles errores de compilación o ejecución.
- **RF-04. Interacción mediante terminal:** Se debe habilitar una terminal emulada, interactiva y segura, que permita al alumno introducir datos y ejecutar comandos en tiempo real para probar sus programas.
- **RF-05. Asistencia conversacional inteligente (Chatbot):** El sistema debe incorporar un Agente IA que asista al alumno. Este tutor virtual debe aplicar estrictamente un enfoque pedagógico socrático para la resolución de dudas, guiando al estudiante mediante preguntas en lugar de proporcionar código resuelto.
- **RF-06. Procesamiento y consulta de documentación (RAG):** El sistema debe permitir la incorporación de archivos PDF docentes, vectorizando su información para que la IA la utilice como contexto. Esto permitirá al alumno realizar consultas sobre la teoría en lenguaje natural y obtener respuestas fundamentadas en el temario oficial.

3.3.2. Requisitos No Funcionales

Los requisitos no funcionales definen los atributos de calidad, restricciones tecnológicas, seguridad y rendimiento de la plataforma:

- **RNF-01. Seguridad, aislamiento y límite de recursos (*Sandboxing*):** Para garantizar que la ejecución de programas de terceros no comprometa el sistema ni acceda a información de otros usuarios, la compilación y ejecución debe realizarse en contenedores Docker estrictamente aislados. Estos carecerán de conexión a red y tendrán límites físicos predefinidos de memoria (256 MB) y tiempo de ejecución (`timeout`).
- **RNF-02. Rendimiento, baja latencia y asincronía:** La interfaz debe mantenerse operativa y fluida en todo momento. Las respuestas del asistente inteligente deben procesarse de forma asíncrona mediante flujos de datos (*streams*) para garantizar que la interfaz de usuario no se bloquee mientras se espera la respuesta completa del LLM.
- **RNF-03. Usabilidad y diseño adaptativo (*Responsive*):** La aplicación debe proporcionar una experiencia de uso intuitiva. La interfaz gráfica debe adaptarse correctamente a resoluciones de escritorio y portátiles, priorizando la organización visual de los elementos para reducir la carga y saturación cognitiva del estudiante.
- **RNF-04. Escalabilidad:** La arquitectura de *backend*, desarrollada en FastAPI, debe permitir atender a múltiples usuarios concurrentes manteniendo tiempos de respuesta adecuados. Para ello, gestionará un *Warm Pool* de contenedores precargados que permita absorber picos de demanda sin penalizar el tiempo de arranque de los entornos de ejecución.
- **RNF-05. Disponibilidad y fiabilidad:** El sistema debe gestionar correctamente errores internos, fallos de ejecución de código o cortes de comunicación, garantizando la continuidad del servicio, la recuperación rápida de los entornos (contenedores efímeros) y la integridad de la información del usuario.

3.4. Planificación Temporal

El desarrollo del proyecto se ha estructurado en cinco fases principales, abarcando desde la investigación preliminar hasta la redacción de la memoria. El volumen total estimado de trabajo asciende a 300 horas, distribuidas de la siguiente manera:

- **Fase 1: Investigación y Diseño Arquitectónico (40 h):** Estudio de viabilidad tecnológica, evaluación de las APIs de modelos de lenguaje (OpenAI, Gemini, Groq) y diseño de la arquitectura del aislamiento de procesos.
- **Fase 2: Prototipado y Desarrollo del Frontend (50 h):** Creación de la interfaz de usuario en React. Esta fase inicial permitió asentar la experiencia de usuario (UX/UI) y construir los componentes visuales base (el editor de código, la caja del chat y la consola) que posteriormente guiarían el desarrollo de la lógica del servidor.
- **Fase 3: Desarrollo del Backend y Motor de Ejecución (70 h):** Implementación de la API REST en FastAPI, desarrollo de la lógica de compilación remota y creación del sistema *Warm Pool* mediante Docker para asegurar el aislamiento (*sandboxing*) de los códigos enviados desde la interfaz.
- **Fase 4: Ecosistema IA, RAG y Comunicación Asíncrona (80 h):** Integración del Agente IA con sus respectivos *prompts* pedagógicos y desarrollo del módulo de lectura de apuntes (RAG). Paralelamente, se implementó el Analizador de Perfiles y se estableció la conexión bidireccional por WebSockets para la pseudo-terminal de Linux.
- **Fase 5: Pruebas, Refinamiento y Documentación (60 h):** Fase de iteración final donde se corrigieron errores de integración, se automatizaron las pruebas de seguridad con `pytest` y despliegue con Github Actions, se realizaron las pruebas de usabilidad con alumnos y se redactó la memoria del Trabajo de Fin de Grado.

3.5. Presupuesto

Aunque este proyecto se ha desarrollado en el marco académico, resulta conveniente realizar una estimación del coste que supondría su desarrollo y despliegue en un entorno empresarial. El presupuesto se divide en tres partes fundamentales: recursos humanos, infraestructura de hardware y licencias de software (APIs).

3.5.1. Estimación del coste de desarrollo

Si la Universidad de Granada o una institución privada decidiera adoptar TutorIA de forma oficial para dar soporte a cientos de alumnos concurrentes, las capas gratuitas resultarían insuficientes debido a las restricciones de ancho de banda, tasa de peticiones y capacidad de cómputo.

Concepto	Detalles y Recursos Necesarios	Coste Estimado
Desarrollo Inicial		
Honorarios	Ingeniero Junior (300 h a 13 €/h)	3.900,00 €
Mantenimiento Mensual		
Servidores	Google Cloud Platform (Instancia e2-standard-4)	120,00 € / mes
Base de Datos	Supabase Pro	25,00 € / mes
APIs de IA	Consumo de tokens estimado (100 alumnos)	80,00 € / mes
Infraestructura Web	Vercel Pro	20,00 € / mes
Seguridad y Dominio	Certificados SSL y Proxy comercial	4,16 € / mes
Total Mensual		**249,16 € / mes**

Cuadro 3.1: Estimación de costes para un entorno empresarial en producción.

A continuación se presenta un presupuesto estimado del coste de desarrollo inicial y el coste mensual en un entorno de producción empresarial.

Como se observa en la estimación, el coste más significativo reside en el desarrollo inicial del software. Una vez en producción, el mantenimiento de la plataforma rondaría los 250,00 € mensuales, una cifra competitiva considerando la escalabilidad de la arquitectura y el valor aportado.

3.5.2. Consideraciones Éticas y Privacidad

El desarrollo de software educativo conlleva una responsabilidad respecto al tratamiento de los datos de los estudiantes. Dado que TutorIA actúa como un intermediario entre el alumno y modelos de inteligencia artificial de terceros (Groq, Google Gemini), es importante analizar el flujo de la información para garantizar el cumplimiento del Reglamento General de Protección de Datos (RGPD).

En la arquitectura actual, el código fuente escrito por los alumnos y sus consultas teóricas son enviados a servidores externos para su procesamiento semántico. Para mitigar los riesgos asociados a la privacidad, el sistema se ha diseñado bajo el principio de minimización de datos. Tal y como se detalla en el diseño de la arquitectura, la plataforma permite un uso en "modo anónimo". Este enfoque *stateless* permite a los estudiantes interactuar con el compilador y el tutor socrático sin necesidad de proporcionar correos electrónicos, nombres o credenciales institucionales, garantizando que los fragmentos de código compartidos con las APIs externas no puedan ser asociados a una identidad real de forma directa.

No obstante, como línea de trabajo futuro y para una hipotética implantación oficial y obligatoria en la Universidad de Granada, se debería

contemplar la sustitución de estas APIs de terceros por modelos de lenguaje de código abierto (como Llama 3 o Mistral) alojados localmente en los propios servidores de la institución, asegurando así un control tecnológico y de datos absoluto.

Capítulo 4

Diseño e Implementación del Sistema

4.1. Introducción

En este capítulo se detalla la arquitectura de software, las decisiones de diseño y las tecnologías implementadas para la construcción de *TutorIA*. El objetivo principal ha sido crear una plataforma interactiva, segura y escalable, capaz de integrar modelos de lenguaje con entornos de ejecución de código aislados y en tiempo real. A lo largo de las siguientes secciones, se desglosará el funcionamiento del modelo general, el ecosistema multi-agente, la gestión de contenedores efímeros y la estructura de la base de datos que da soporte a todo el entorno.

4.2. Visión General de la Arquitectura

El sistema ha sido diseñado siguiendo un patrón Cliente-Servidor, ampliado con componentes de procesamiento asíncrono y comunicación bidireccional continua (WebSocket). La aplicación no solo devuelve texto, sino que emula una terminal interactiva y orquesta la compilación de código fuente mediante Inteligencia Artificial, dividiéndose en cuatro grandes bloques funcionales que se comunican e interactúan entre sí.

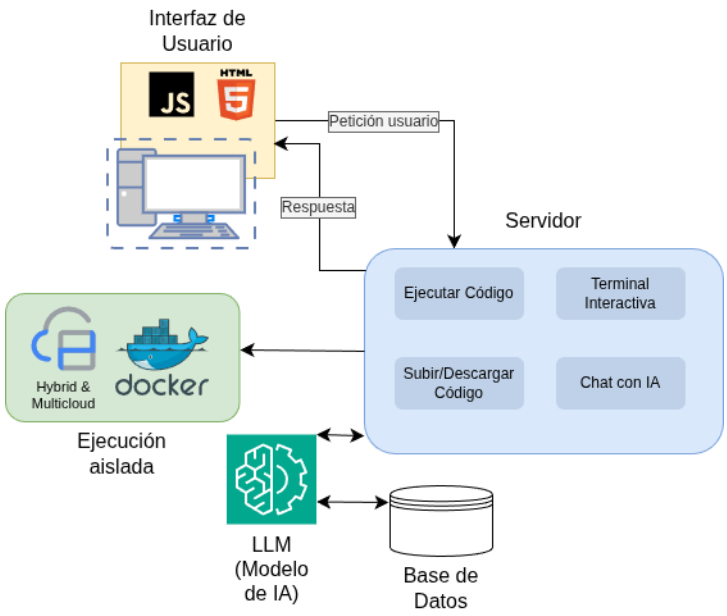


Figura 4.1: Diagrama General de la Arquitectura del Sistema

Como se puede observar en la figura anterior, el flujo de información y responsabilidades se distribuye de la siguiente manera:

- **Capa de Presentación (*Frontend*):** Es la interfaz directa con el estudiante (Cliente). Esta capa no realiza lógica compleja, sino que actúa como una interfaz reactiva. Mantiene conexiones HTTP estándar para el envío de mensajes al chat y la gestión de archivos, al mismo tiempo que establece un túnel bidireccional (WebSocket) acoplado a la librería Xterm.js para renderizar la pseudo-terminal de Linux en tiempo real.

- **Capa de Infraestructura y Red (Google Cloud):** Para garantizar la disponibilidad y el despliegue continuo, el sistema se aloja en una Máquina Virtual en la nube. En este nivel, un flujo de CI/CD automatiza los despliegues, mientras que un gestor de procesos (*Process Manager*) garantiza la resiliencia del servidor. Un túnel proxy expone de forma segura los puertos locales hacia el exterior mediante HTTPS y WSS (WebSockets Seguros).

- **Capa Lógica (*Backend*):** Es el núcleo inteligente del sistema. Un limitador de peticiones protege los enrutadores (*routers*) contra ataques o saturación. En esta puerta de entrada se implementa un mecanismo de autenticación opcional y *stateless* (no guarda información o registros): mediante la inspección de las cabeceras HTTP, el sistema intenta

extraer y validar un *JSON Web Token* (JWT). Si el *token* es válido, se recupera el identificador del estudiante para inyectar su perfil formativo en el contexto de la IA. Si el token no existe, el sistema asume un modo anónimo, garantizando el acceso a la tutorización pero sin persistencia de perfilado. Esta capa orquesta dos grandes sub-sistemas: por un lado, el subsistema de emulación (*pty* y gestión del gestor de contenedores asíncrono) y, por otro, el ecosistema de Inteligencia Artificial. Este último aloja al agente principal (el tutor socrático), al servidor de herramientas MCP, y a un agente analista secundario que opera en segundo plano.

- **Capa de Ejecución y Persistencia (Docker y BD):** Todo el código y las interacciones de terminal se ejecutan en un entorno fuertemente aislado (*sandboxing*) dentro de contenedores gestionados dinámicamente (*Warm Pool*). En paralelo, la persistencia de datos recae sobre una base de datos relacional PostgreSQL, que no solo maneja la autenticación, sino que aloja el índice vectorial necesario para recuperar el contexto semántico de los apuntes y diapositivas, así como del perfil del alumno.

Esta separación de capas permite que el sistema sea modular: el *frontend* desconoce la existencia de los contenedores para la ejecución de código, y los modelos de Inteligencia Artificial interactúan con la base de datos y la compilación únicamente a través de las herramientas del servidor MCP, manteniendo un alto nivel de seguridad.

4.2.1. Stack Tecnológico y Justificación

Para llevar a cabo los conceptos teóricos descritos anteriormente y construir una arquitectura robusta, escalable y en tiempo real, la elección del *stack* tecnológico resulta crítica. En el diseño de *TutorIA*, cada herramienta ha sido seleccionada con un propósito específico para resolver las necesidades del ecosistema:

- **React y Vite (*Frontend*):** En la capa de presentación se ha optado por usar React (con TypeScript) y el empaquetador Vite [27]. Esta combinación permite un renderizado ultrarrápido sin necesidad de recargar la página, manteniendo el estado de la aplicación gracias a su actualización en tiempo real (HMR - *Hot Module Replacement*). El uso de React resulta vital para manejar el estado complejo de una interfaz que debe actualizar simultáneamente, y sin bloqueos, el chat asíncrono, el editor de texto y la emulación de la terminal.

- **FastAPI (*Backend*):** Se eligió este *framework* de Python por su alto rendimiento y su soporte nativo para la programación asíncrona (*async/await*) [28]. En un proyecto multi-agente, el servidor debe realizar múltiples operaciones de entrada/salida de forma concurrente, como consultar la base de datos, esperar respuestas del LLM o comunicarse con el demonio de Docker. Además, FastAPI facilita enormemente la gestión de WebSockets y proporciona un tipado fuerte que mejora la seguridad en la validación de datos.

- **Supabase (PostgreSQL y pgvector):** Para la persistencia de datos se ha seleccionado Supabase, una alternativa *open-source* a Firebase basada en PostgreSQL [29]. Su elección se fundamenta en la integración nativa de la extensión `pgvector`, esencial para el almacenamiento de *embeddings* y la recuperación de información del motor RAG. Asimismo, proporciona un sistema de autenticación seguro basado en JWT, permitiendo una gestión de sesiones *stateless*.

- **Docker:** Herramienta seleccionada para implementar el *sandboxing*. Permite encapsular el código del alumno y ejecutarlo en contenedores efímeros aislados del sistema anfitrión, garantizando la seguridad perimetral de la aplicación frente a ejecuciones maliciosas o errores críticos [30].

- **LangGraph y FastMCP:** Para la orquestación del agente inteligente, LangGraph permite modelar el flujo de la conversación y las decisiones del LLM como un grafo de estados cíclicos. Esto ofrece un control determinista sobre el flujo del agente, permitiendo forzar a la IA a pasar por la herramienta de compilación antes de contestar al alumno. FastMCP se emplea para estandarizar la conexión entre este agente y las herramientas locales del servidor, adhiriéndose al estándar MCP [22].

- **Vercel, Google Cloud, Ngrok y PM2 (Infraestructura):** Para el despliegue, Vercel distribuye el *frontend* globalmente con baja latencia [31]. El *backend*, debido a sus requisitos de acceso a bajo nivel para Docker, se aloja en una Máquina Virtual de Google Cloud [32]. PM2 actúa como gestor de procesos para garantizar la resiliencia del servidor ante caídas [33], y Ngrok expone el servicio local mediante túneles seguros (HTTPS/WSS) solucionando problemas de red y certificados [34].

4.2.2. Ciclo de vida de una interacción

Para comprender cómo se integran los componentes descritos, el ciclo de vida de una consulta del alumno en *TutorIA* sigue un flujo de control basado

en el paradigma *ReAct* (*Reasoning and Acting*):

1. **Entrada y Contextualización:** El alumno envía una consulta. El servidor recibe el mensaje y, utilizando el token de sesión, inyecta el perfil del alumno (fortalezas/carencias) y el contexto actual del proyecto en el estado gestionado por *LangGraph*.
2. **Fase de Razonamiento (*Thought*):** El orquestador inicia el bucle, el LLM analiza el estado actual y genera una reflexión interna: ¿necesito ejecutar código? ¿necesito consultar apuntes? ¿puedo responder directamente?
3. **Fase de Acción (*Action*) vía MCP:** Si el agente decide usar una herramienta, invoca el servidor MCP. Este servidor, actuando como un puente seguro ejecuta la lógica (compilar o buscar en la BD) y devuelve el resultado al agente.
4. **Fase de Observación:** El agente recibe el resultado (la salida de la compilación o el contenido de los apuntes) y actualiza su razonamiento para formular la respuesta socrática precisa.
5. **Perfilado Asíncrono:** Una vez que el tutor responde, el *backend* dispara una tarea en segundo plano donde el Agente Analista evalúa la interacción, actualizando las debilidades del estudiante en la base de datos sin afectar a la latencia del chat.

4.3. Capa de Presentación: Interfaz del Usuario (UI)

Esta capa representa la interfaz directa con el estudiante. El principal objetivo es ofrecer un entorno de trabajo unificado que evite el cambio de contexto. Históricamente, se ha debido de alternar entre un editor de código, una terminal de comandos y un navegador web para buscar dudas, pero la interfaz de *TutorIA* los unifica en una sola pantalla dividida en paneles interactivos.

Esta distribución, con el editor de código integrado a un lado y el chat del tutor socrático y la terminal al otro, permite al estudiante visualizar su progreso, compilar y recibir la retroalimentación del asistente de forma simultánea, manteniendo el foco absoluto en la resolución del problema sin abandonar la aplicación.

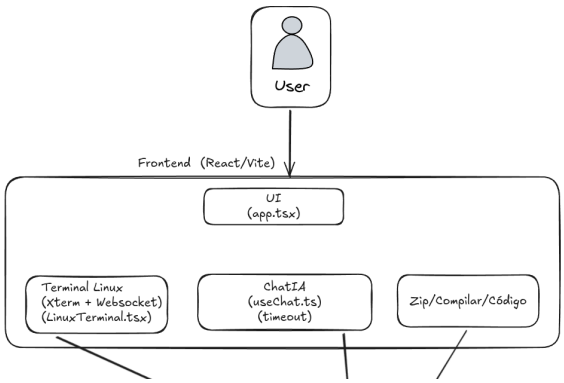


Figura 4.2: Detalle de la Capa de Presentación (*Frontend*).

4.4. Capa Lógica y Ecosistema Multi-Agente

El núcleo de la capa lógica de *TutorIA* abandona el enfoque de un único modelo monolítico para implementar un ecosistema multi-agente, estableciendo una jerarquía de responsabilidades en tres pilares:

- **La Lógica de los Agentes:** El sistema orquesta dos entidades diferenciadas. Por un lado, un tutor principal maneja la interacción directa con el alumno. Por otro, un agente analista secundario opera asíncronamente en segundo plano, evaluando de forma silenciosa el progreso del estudiante para adaptar el nivel de dificultad sin penalizar la latencia del chat.
- **La Orquestación (*LangGraph*):** Para evitar que el agente actúe como una caja negra probabilística, el flujo de la conversación se controla mediante grafos de estado bajo el paradigma *ReAct* (*Reasoning and Acting*) gracias a *LangGraph*. Fuerza al modelo a generar una cadena de razonamiento (*Thought*) ante cada consulta, evaluando (*Observation*) si dispone de información suficiente para responder mediante preguntas socráticas, o si necesita pausar la generación para invocar herramientas externas (*Action*).
- **La Capa de Ejecución Aislada (MCP):** El *Model Context Protocol* (MCP) actúa como el puente de comunicación entre la inteligencia del sistema y el entorno real. Gracias a este estándar, el agente desconoce los detalles de implementación, simplemente solicita acciones (como compilar código en un contenedor Docker o buscar en el temario) y el protocolo se encarga de ejecutarlas de forma estricta y segura.

Esta integración permite que *TutorIA* funcione como un sistema dinámico. El orquestador (*LangGraph*) ejecuta un flujo donde el agente piensa,

actúa y observa, invocando herramientas expuestas a través de MCP. Este diseño permite que el razonamiento del modelo y la ejecución técnica (como la compilación) se mantengan separados, facilitando que el sistema sea fácil de ampliar o actualizar sin necesidad de reescribir toda la lógica interna [14].

4.4.1. El Agente Principal: Tutor Socrático

El agente principal, es decir, el "Tutor", actúa como el cerebro del sistema. Su responsabilidad es procesar cada interacción del estudiante para dirigir el flujo de la aplicación. En lugar de limitarse a generar texto estático, modela la conversación como un grafo de estados. Antes de enviar la petición al LLM, el orquestador recupera la memoria de los mensajes anteriores de la sesión e inyecta el perfil de debilidades del alumno extraído de la base de datos y el estado actual del proyecto (código en el editor y terminal).

Cuando entra un mensaje, el agente determina si cuenta con el contexto suficiente para formular una respuesta empleando el método socrático, o si necesita realizar llamadas a herramientas externas para validar la información. Para asegurar que la IA adopte el rol de tutor y nunca resuelva el ejercicio directamente, se configura un "System Prompt" estricto que se concatena de manera invisible al inicio de cada evaluación.

Listing 4.1: Configuración del System Prompt del agente

```
# Prompt de sistema
texto_perfil = f"""
    === PERFIL FORMATIVO DEL ALUMNO (PERSONALIZACION) ===
    - Fortalezas: {fortalezas}
    - Carencias/Debilidades: {carencias}

    INSTRUCCION PEDAGOGICA: Utiliza esta informacion para ajustar tu
        nivel de dificultad.
    Si tiene carencias en un concepto que aparece en su codigo, se mas
         incisivo con preguntas socraticas sobre ese tema.
    Si domina algo, no te detengas demasiado ahi.
    """
rol_sistema = f"Eres un tutor socratico de 'Metodologia de la
    Programacion' en C/C++ de la Universidad de Granada (UGR).\n{
    texto_perfil}"
    reglas_especificas = """
        REGLAS ESTRICTAS (C++):
        0. CONVERSACION: Si el alumno solo saluda, responde
            normalmente SIN herramientas.
        1. BUSQUEDA OBLIGATORIA: Para teoria, usa 'buscar_apuntes_ugr
            '.
        2. CITAS CLICABLES: Tu respuesta DEBE EMPEZAR con el enlace [
            texto](url).
        3. ANTI-BUCLE: Responde a la ULTIMA pregunta.
        4. COMPILADOR: Usa 'compilar_cpp' SOLO si hay errores.
        5. CIERRE: Termina siempre con una pregunta socratica."""
        texto_terminal = ""
```

4.4.2. El Agente Secundario: Analista Asíncrono

Para que el tutor personalice sus respuestas, el sistema debe analizar las carencias del alumno. Sin embargo, procesar esta evaluación con un LLM consume tiempo, penalizando la experiencia de usuario. Para no paralizar el chat del alumno, evitando cuellos de botella y manteniendo la fluidez de la interfaz, se implementó un patrón de concurrencia: la evaluación se delega a una *Background Task* (tarea en segundo plano), permitiendo que, tan pronto como el tutor genere su respuesta, el servidor libere la conexión HTTP y devuelva el mensaje al usuario.

El agente secundario analiza la conversación en segundo plano mientras el alumno sigue interactuando. Dado que la IA a veces devuelve la respuesta envuelta en bloques de Markdown (texto sucio), el *backend* limpia la cadena mediante expresiones regulares para extraer el JSON válido. Finalmente, como las políticas RLS impiden que un usuario modifique su propio perfil de evaluación directamente, el servidor utiliza la clave "Service Role"de la base de datos para elevar privilegios y guardar la actualización de forma segura.

Listing 4.2: Tarea asíncrona de análisis de perfil

```
async def update_profile_background(user_id: str,
    conversation_text: str):
   last_msg = messages[-1]['content']

    prompt = f"""
    Eres un evaluador pedagogico. Tu mision es detectar QUE NO
         SABE el alumno y QUE DOMINA.
    CONVERSACION: {last_msg}

    INSTRUCCIONES:
    1. Si el alumno admite no saber algo, registralo en '
        carencia'.
    2. Si el alumno NO demuestra dominar nada en este mensaje,
         el valor de 'fortaleza' DEBE SER el anterior pasado.
        No inventes datos.
    3. Se breve (maximo 5 palabras por campo).

    Responde SOLO con este formato JSON: {{"carencia": "...",
        "fortaleza": "..."}}
    """
    # Limpia Markdown
    content = content.replace("```json", "").replace("```", ""
        ).strip()

    parsed_data = json.loads(content)

    # Actualiza con admin
    supabase_client.table("perfiles").update({
        "fortalezas": data.get('fortaleza', 'Ninguna detectada
            '),
        "carencias": data.get('carencia', 'Ninguna detectada')
    }).eq("id", user_id).execute()
```

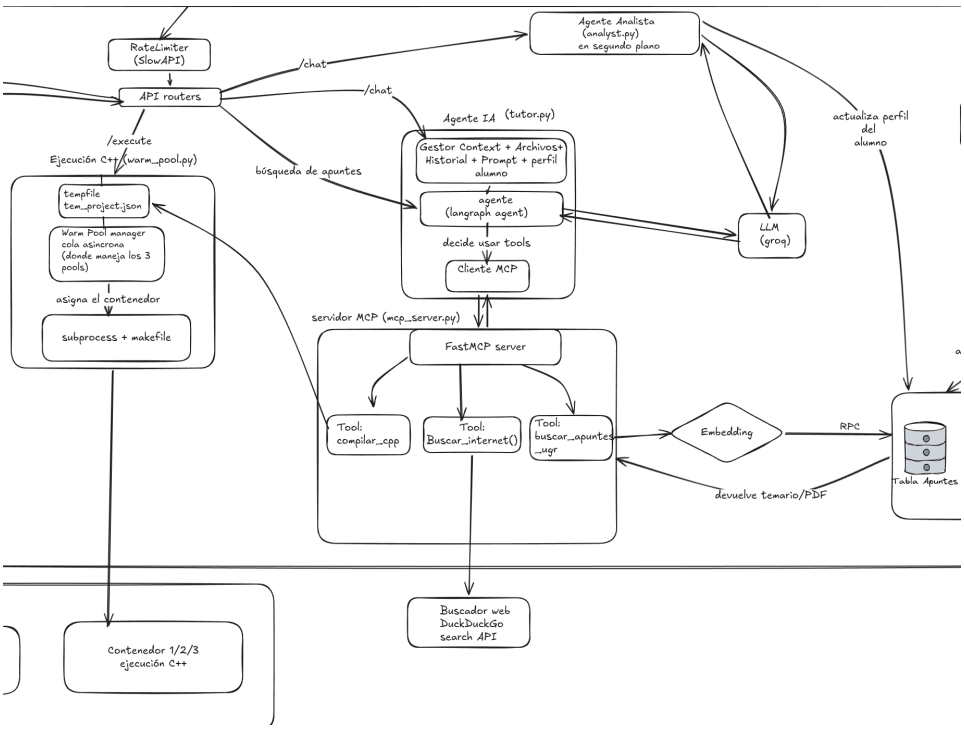


Figura 4.3: Detalle del ecosistema Multi-Agente y conexión MCP.

4.4.3. Ecosistema de Herramientas MCP

Para dotar al Tutor Socrático de capacidades de interacción con el entorno que salga de la simple generación de texto, se ha implementado una arquitectura basada en el estándar *Model Context Protocol* (MCP). Este enfoque fomenta una estricta separación de responsabilidades: el Gran Modelo de Lenguaje (LLM) actúa exclusivamente como motor de razonamiento semántico, mientras que la ejecución de tareas se delega a un ecosistema de herramientas especializadas.

La implementación técnica se ha desarrollado permitiendo exponer funciones locales al agente de IA. Mediante el uso de decoradores y tipado estricto (*type hints*), el sistema genera dinámicamente un esquema JSON que se envía al LLM como parte de su configuración inicial, siendo este esquema como un "contrato" indicando a la IA qué funciones existen, qué parámetros requieren y qué hacen cada una. Cuando el modelo decide ejecutar una herramienta, envía un objeto JSON que el servidor parsea para ejecutar la función correspondiente. El resultado se devuelve al LLM empaquetado como un nuevo mensaje de sistema. El ecosistema diseñado para *TutorIA* se compone de tres herramientas fundamentales:

- **Motor RAG (`buscar_apuntes_ugr`):** Constituye la fuente de verdad. Permite al agente consultar la base de datos vectorial para fundamentar sus explicaciones en el temario oficial.
- **Entorno de Ejecución (`compilar_cpp`):** Habilita al agente para invocar el contenedor Docker y compilar el código del alumno de forma autónoma o cuando el alumno se lo indica. Esto permite a la IA analizar los errores de la salida estándar (*stdout*) o de error (*stderr*) del compilador `gcc` antes de formular su respuesta pedagógica.
- **Búsqueda Web en Tiempo Real (`buscar_internet`):** Actúa como un sistema de respaldo (*fallback*). Si el motor RAG no obtiene resultados con la suficiente similitud semántica para una consulta, es decir, conceptos que no estén explícitamente añadidos en el temario o incluso errores muy complejos, el modelo tiene instrucciones de utilizar una API de búsqueda web. Esta herramienta permite al sistema acceder a información actualizada de internet, mitigando el riesgo de alucinación.

Esta orquestación multi-herramienta convierte a *TutorIA* en un sistema dinámico capaz de auditar, verificar y fundamentar sus propias respuestas antes de presentarlas al usuario final.

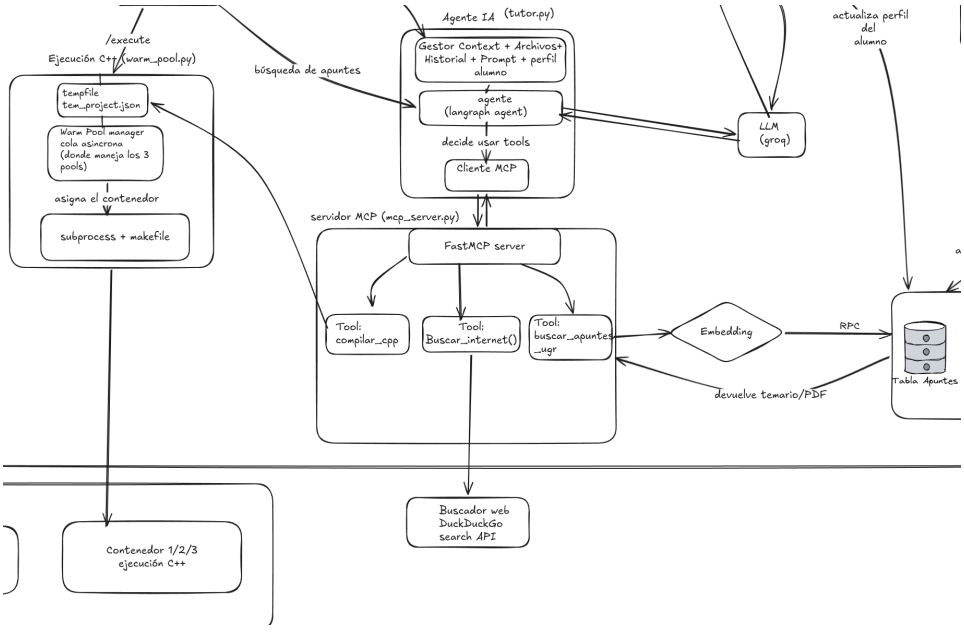


Figura 4.4: Detalle del Servidor MCP y herramientas expuestas.

4.5. Integración de Ejecución Remota (Docker)

4.5.1. Aislamiento de código con Docker

La ejecución de código de terceros (estudiantes) es un vector de ataque crítico. Se previene cualquier ataque de inyección o bucle infinito aislando la compilación en contenedores Docker efímeros. Para evitar la latencia de arranque (*Cold Start*), el *backend* gestiona un *Warm Pool* de contenedores pre-iniciados. Al recibir una petición, se asigna el contenedor libre, se compila el código C++, y el entorno se destruye o limpia automáticamente.

Listing 4.3: Instanciación de contenedor Docker

```
import asyncio
import subprocess

class WarmPoolManager:
    def __init__(self, pool_size=3):
        self.pool_size = pool_size
        self.queue = None
        self.containers = [f"tutor_warm_{i}" for i in range(pool_size)]

    async def start_pool(self):
        print(f" Iniciando Warm Pool con {self.pool_size} contenedores...")
        # Inicializamos la cola aqui para que use el loop de la app actual
        self.queue = asyncio.Queue()

        for name in self.containers:
            # Limpieza preventiva
            subprocess.run(["docker", "stop", name], capture_output=True)

            cmd = [
                "docker", "run", "-d", "--rm",
                "--name", name,
                "--network", "none",
                "--memory", "256m",
                "--memory-swap", "256m",
                "--cpus", "0.5",
                "--pids-limit", "64",
                "--security-opt", "no-new-privileges",
                "--cap-drop", "ALL",
                "tutor-ugr-image:latest",
                "tail", "-f", "/dev/null"
            ]
            subprocess.run(cmd, check=True)
            await self.queue.put(name)

        # Tiempo para que Docker estabilice el sistema de archivos
        await asyncio.sleep(1.5)
        print(" Warm Pool listo.")

    async def stop_pool(self):
        print(" Deteniendo Warm Pool...")
        for name in self.containers:
```

```
42              subprocess.run(["docker", "stop", name], capture_output=
                    True)
43
44          if self.queue:
45              while not self.queue.empty():
46                  try:
47                      self.queue.get_nowait()
48                  except asyncio.QueueEmpty:
49                      break
50
51      async def get_container(self):
52          if self.queue is None:
53              raise RuntimeError("El pool no esta inicializado.")
54          return await self.queue.get()
55
56      def return_container(self, name):
57          if self.queue is not None:
58              self.queue.put_nowait(name)
59
60  pool = WarmPoolManager(pool_size=3)
```

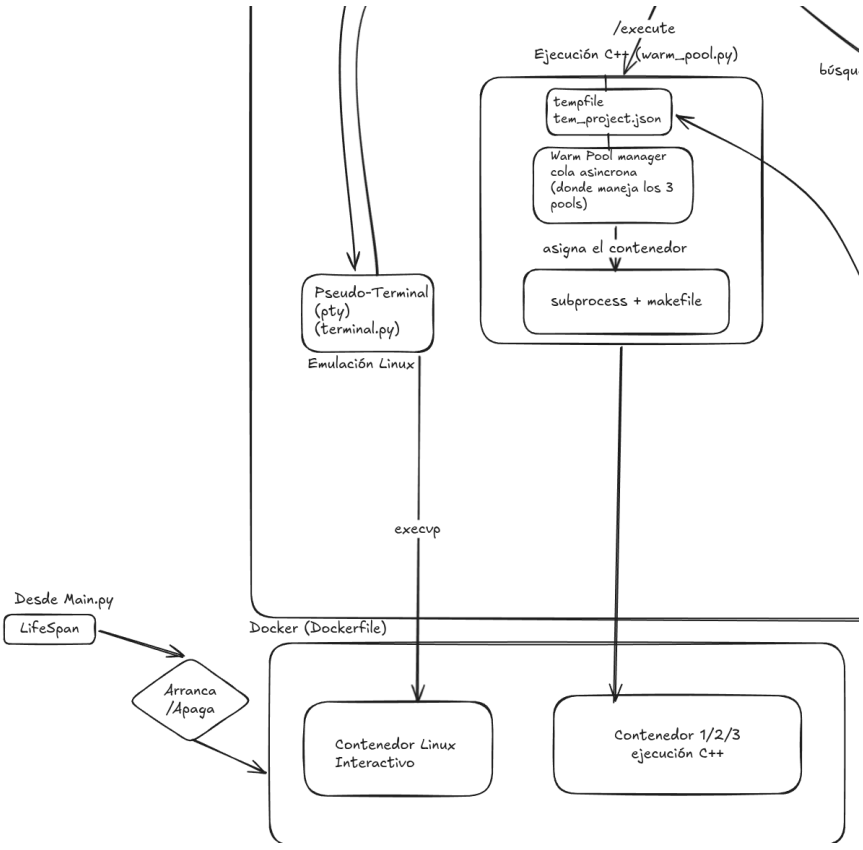


Figura 4.5: Detalle del bloque de Ejecución C++ y gestión de contenedores.

4.5.2. Comunicación bidireccional mediante WebSockets

El protocolo HTTP clásico sigue un modelo estricto de "petición-respuesta", lo que resulta muy ineficiente para transmitir la salida de una terminal en ejecución, ya que obligaría al cliente a preguntar constantemente al servidor si hay nuevos datos (*Long Polling*). Para solucionarlo, se implementa el estándar WebSockets [35], que establece una conexión TCP persistente, bidireccional y en tiempo real entre el servidor pty (pseudo-terminal) y la librería Xterm.js del cliente.

Listing 4.4: Conexión de pseudo-terminal mediante WebSocket

```
@app.websocket("/ws/terminal")
async def terminal_endpoint(websocket: WebSocket):
    @router.websocket("/ws/terminal")
async def websocket_terminal(websocket: WebSocket):
    await websocket.accept()

    # Creamos una Pseudo-Terminal (PTY) de Linux
    master_fd, slave_fd = pty.openpty()

    .........

    async def read_from_pty():
        loop = asyncio.get_running_loop()
        try:
            while True:
                # Leemos lo que dice Linux
                data = await loop.run_in_executor(None, os.read,
                    master_fd, 4096)
                if not data:
                    break
                # Lo mandamos por el WebSocket
                await websocket.send_text(data.decode("utf-8", "
                    replace"))
        except Exception:
            pass


    # Tarea para ESCUCHAR a React y meterlo en Bash
    async def write_to_pty():
        try:
            while True:
                data = await websocket.receive_text()
                # Escribimos las teclas del alumno en Linux
                os.write(master_fd, data.encode("utf-8"))
        except WebSocketDisconnect:

            pass
```

4.6. Modelo de Datos y Motor RAG (PostgreSQL y Supabase)

4.6.1. Manejo de Sesiones Stateless y Políticas RLS

La persistencia del sistema se gestiona de forma centralizada en una base de datos PostgreSQL. En un entorno educativo diseñado para soportar múltiples usuarios concurrentes el manejo tradicional de sesiones basadas en el servidor (almacenando el estado en memoria) resulta ineficiente. Para resolver este problema, la arquitectura de *TutorIA* implementa un modelo de autenticación *stateless* (sin estado) utilizando JSON Web Tokens (JWT) [36].

En lugar de acoplar la autenticación a cada ruta de forma manual, se ha diseñado un inyector de dependencias que actúa como *middleware* de seguridad. Este módulo intercepta las peticiones HTTP, extrae la cabecera de autorización y aísla el *token*. Al dejar la verificación directamente a la base de datos, el *backend* confirma la identidad del estudiante sin necesidad de mantener sesiones activas en el servidor.

Destaca en este diseño su opcionalidad (`Header(None)`), es decir, si un usuario accede sin credenciales, el sistema no rechaza la petición, sino que asume un rol anónimo. Esto garantiza la accesibilidad a la herramienta de tutorización para pruebas rápidas, reservando el almacenamiento del perfilado adaptativo únicamente para los usuarios verificados.

Listing 4.5: Middleware de autenticación stateless y extracción de JWT

```
1         from fastapi import Header
2         from supabase import create_client
3         import os
4
5         supabase = create_client(os.getenv("SUPABASE_URL"), os.getenv(
              "SUPABASE_KEY"))
6
7         async def get_optional_user(authorization: str = Header(None))
              :
8         if not authorization:
9         return None
10
11        # Extrae token JWT del header
12        token = authorization.replace("Bearer ", "")
13        try:
14            # Verifica token en Supabase y devuelve ID
15            user = supabase.auth.get_user(token)
16            return user.user.id
17        except:
18            return None
```

Una vez verificada la identidad del estudiante, la seguridad de los datos se garantiza en la propia capa de la base de datos mediante políticas RLS (Row Level Security). Cuando el *backend* realiza una consulta autenticada, PostgreSQL evalúa estas políticas validando que el identificador del usuario (`user.id`) coincide de forma estricta con la columna del registro. Este enfoque desplaza la responsabilidad de la autorización desde la API hacia la base de datos, impidiendo que un alumno pueda acceder a las conversaciones o datos de otro compañero por una vulnerabilidad en el código.

4.6.2. Procesamiento y vectorización de apuntes (RAG)

Para dotar al tutor de conocimiento oficial, evitar que sufra de "alucinaciones" y asegurar que las respuestas se basen en el temario y criterio oficial de la universidad, se ha implementado un motor RAG. A nivel de implementación, el flujo de trabajo se divide en dos fases bien diferenciadas: la inyección (*Ingestion*) y la recuperación (*Retrieval*).

- **1. Inyección y Limpieza de Datos (*Ingestion*):** Para procesar los PDFs oficiales (por ejemplo, de la asignatura de Metodología de la Programación), se ha desarrollado un script en Python que utiliza la librería PyPDF. Dado que el texto extraído directamente de un PDF suele contener ruido, el script aplica funciones de limpieza mediante expresiones regulares antes de fragmentar el contenido (*chunking*).

  Cada fragmento de texto limpio se procesa a través de una API para obtener su representación vectorial (*embedding*) en múltiples dimensiones. Finalmente, estos vectores se almacenan en una base de datos relacional apoyándose en una extensión (*pgvector*) para poder indexar campos matemáticos.

  Listing 4.6: Flujo de limpieza e inyección vectorial de apuntes para los apuntes de la asingatura de Metodogía de la Programación

```
1         def limpiar_ruido(texto: str) -> str:
2         # Elimina cabeceras estandar de las diapositivas
3         ruido_conocido = ["METODOLOGIA DE LA PROGRAMACION", "
              GRADO EN INGENIERIA", "1531"]
4         for ruido in ruido_conocido:
5         texto = texto.replace(ruido, "")
6
7         # Elimina trazas del entorno de ejecucion
8         lineas_limpias = [l.strip() for l in texto.split('\n')
9                           if "process returned" not in l.lower()]
10
11        return re.sub(r'\n{2,}', '\n', "\n".join(lineas_limpias))
              .strip()
12        def ingest_slides(file_path: str, tema: str):
13        reader = PdfReader(file_path)
14        for num_pag, page in enumerate(reader.pages, start=1):
```

```
15          texto_raw = page.extract_text()
16          if not texto_raw: continue
17
18              texto_limpio = limpiar_ruido(texto_raw)
19
20              # Genera el vector multidimensional con Gemini
21              result = client.models.embed_content(
22                  model="gemini-embedding-001",
23                  contents=texto_limpio,
24                  config=types.EmbedContentConfig(task_type="
                        RETRIEVAL_DOCUMENT")
25              )
26              embedding = result.embeddings[0].values
27
28              # Inserta el vector en la tabla de Supabase (pgvector
                    )
29              supabase.table("apuntes").insert({
30                  "tema": tema,
31                  "contenido": texto_limpio,
32                  "embedding": embedding,
33                  "asignatura": "MP",
34                  "diapositiva": num_pag
35              }).execute()
```

- **2. Recuperación y Generación (*Retrieval*):** Cuando el estudiante pregunta dudas teóricas en la interfaz, la herramienta MCP invoca la función de búsqueda de apuntes. En este momento el sistema convierte la pregunta del alumno en un vector de las mismas dimensiones y realiza una llamada a procedimiento remoto (*RPC*) hacia PostgreSQL.

  La base de datos ejecuta una función de "similitud del coseno", cruzando el vector de la pregunta contra todos los vectores almacenados y recuperando las diapositivas con mayor índice de similitud matemática (por encima de un umbral del 40 por ciento). El *backend* formatea estos apuntes construyendo una URL directa al fragmento del documento original en el *Storage* en la nube, y se inyecta en el contexto del LLM para que este formule su respuesta basándose en esos datos.

Listing 4.7: Busqueda vectorial en la base de datos (Recuperación)

```
1               def search_theory_logic(pregunta: str) -> str:
2               # 1. Transformar la pregunta del alumno a un vector
3               result = client_google.models.embed_content(
4               model="gemini-embedding-001",
5               contents=pregunta,
6               config=types.EmbedContentConfig(task_type="
                    RETRIEVAL_QUERY")
7               )
8               vector_pregunta = result.embeddings[0].values
9
10              # 2. Busqueda por similitud del coseno en Supabase (
                    RPC)
11              respuesta = supabase.rpc(
12                  'match_apuntes',
13                  {
14                      'query_embedding': vector_pregunta,
```

```
15                      'match_threshold': 0.40, # Tolerancia de
                            similitud
16                      'match_count': 3
17                  }
18              ).execute()
19
20              # 3. Formateo de contexto para el LLM y enlace de la
                    diapositiva original
21              mejor_match = respuesta.data[0]
22              diap_mejor = mejor_match.get('diapositiva', 0)
23
24              url_pdf = f"{BASE_URL_STORAGE}/{mejor_match['tema']}#
                    page={diap_mejor}"
25
26              contexto = f"[{mejor_match['tema']} - Diapositiva {
                    diap_mejor}]({url_pdf})\n\n"
27              contexto += "INFORMACION EXTRAIDA PARA TU EXPLICACION
                    :\n"
28              for match in respuesta.data:
29                  contexto += f"- {match['contenido']}\n"
30
31              return contexto
```

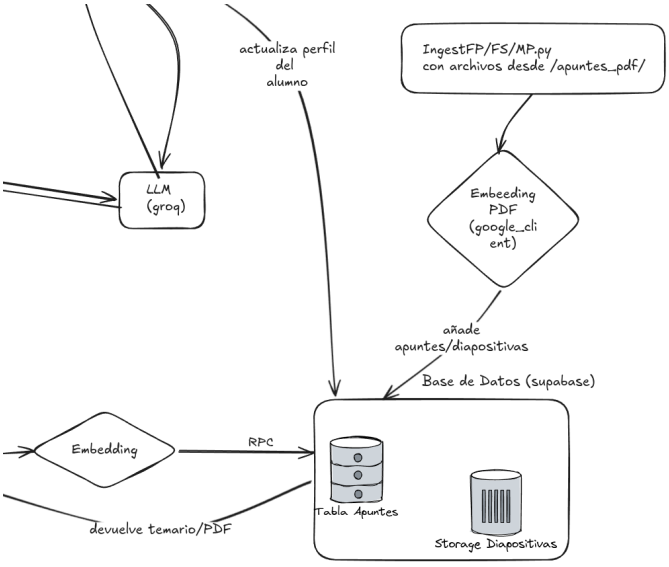


Figura 4.6: Detalle de la Base de Datos Supabase y procesamiento vectorial.

4.7. Despliegue Continuo

El despliegue a producción de *TutorIA* se ha diseñado siguiendo una estrategia de separación de la capa de presentación y la capa lógica. El objetivo es maximizar la disponibilidad, optimizar el rendimiento y automatizar las actualizaciones del sistema mediante flujos de Integración y Despliegue Continuo (CI/CD).

4.7.1. Capa de Presentación: *Edge Network*

El *Frontend* se sirve mediante una plataforma que actúa como un modelo de Plataforma como Servicio (PaaS) optimizado para *frameworks* modernos [31].

Esta elección se justifica por su integración nativa con los repositorios. Cada vez que se realiza un *commit* en la rama principal, el sistema compila automáticamente el proyecto y lo distribuye a través de su *Edge Network* (una red de distribución de contenido global). Esto garantiza que la interfaz de usuario, los recursos estáticos y el entorno de trabajo del estudiante carguen de forma casi instantánea, independientemente de la ubicación desde la que se acceda.

4.7.2. Infraestructura del Backend (IaaS)

El despliegue de la capa lógica presenta un desafío mayor. A diferencia de las APIs web que pueden alojarse en entornos *Serverless* (sin servidor), el *backend* de *TutorIA* tiene requisitos de sistema muy estrictos: necesita controlar el demonio nativo de Docker para reconstruir las imágenes de ejecución y requiere acceso a bajo nivel para abrir subprocesos de pseudo-terminal (*pty*).

Por este motivo, se descartaron soluciones PaaS (*Platform as a Service*) estándar y se optó por un modelo de Infraestructura como Servicio (IaaS), desplegando una Máquina Virtual (VM) en la nube [32]. Este entorno Linux provee el aislamiento y los recursos de memoria necesarios para que la API orqueste la compilación de C++ en los contenedores sin restricciones de entorno.

4.7.3. Automatización del Despliegue (CI/CD)

Para eliminar la intervención manual en el servidor y reducir los tiempos de inactividad, el ciclo de vida del *backend* se ha automatizado mediante flujos de trabajo basados en eventos [37].

Se ha configurado un *pipeline* que se ejecuta automáticamente ante cualquier subida de código (*push*) a la rama de producción y, a través de una conexión segura por SSH, ejecuta un script directamente en la Máquina Virtual que realiza cuatro tareas críticas:

- Sincroniza el repositorio local extrayendo los últimos cambios.
- Reconstruye la imagen base de Docker (tutor-ugr-image) para garantizar que el entorno de aislamiento de los alumnos cuente con las últimas actualizaciones.
- Activa el entorno virtual de Python y actualiza las dependencias.
- Delega el reinicio de los servicios al gestor de procesos.

Listing 4.8: Comandos de daemonización del backend

```
name: Deploy Backend

on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@master
        with:
          host: ${{ secrets.SSH_HOST }}
          username: luisarrabalgutierrez
          key: ${{ secrets.SSH_PRIVATE_KEY }}
          script: |
            cd ~/tutor-ai
            git reset --hard origin/main
            git pull origin main

            # Reconstruir entorno Docker de aislamiento
            cd backend/docker
            docker build -t tutor-ugr-image:latest .

            # Configurar entorno Python
            cd ~/tutor-ai/backend
            [ -d venv ] || python3 -m venv venv
            source venv/bin/activate
            pip install --upgrade pip
            pip install -r requirements.txt

            # Reiniciar servicios
            pm2 restart backend || pm2 start "python3 -m uvicorn
                main:app --host 0.0.0.0 --port 8000" --name
                backend
            pm2 restart ngrok || pm2 start "ngrok http --domain=
                lunar-finicky-concierge.ngrok-free.dev 8000" --
                name ngrok
            pm2 save
```

4.7.4. Gestión de Procesos y Enrutamiento Seguro

Como se observa en la última etapa del flujo, la capacidad del servidor para adaptarse está garantizada por un gestor de procesos avanzado [33]. Si el proceso de la API sufre un error por falta de memoria o una excepción no controlada, se detecta la caída y se reinicia en milisegundos.

Finalmente, la exposición del servicio a internet se securiza mediante túneles inversos [34]. Configurar manualmente un servidor proxy con certificados SSL en una Máquina Virtual puede ser propenso a fallos. Esta solución resuelve el problema estableciendo un túnel encriptado desde la VM hacia servidores públicos, con un dominio preconfigurado y conexiones HTTPS y WSS robustas y seguras para la comunicación en tiempo real con el *frontend*.

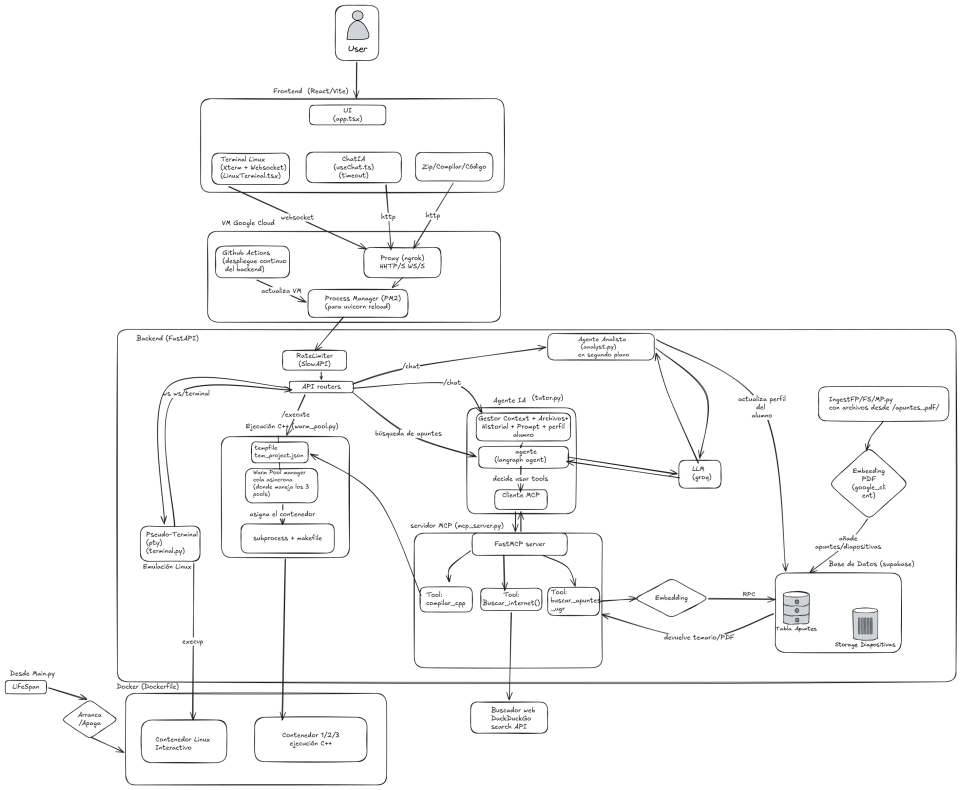


Figura 4.7: Diagrama General Completo de la Arquitectura del Sistema

Capítulo 5

Pruebas y Validación del Sistema

5.1. Introducción

La validación de un entorno interactivo de aprendizaje, especialmente uno que permite la ejecución de código y se apoya en modelos de lenguaje de gran escala (LLMs), requiere una metodología de aseguramiento de la calidad. El objetivo de este capítulo es detallar las diferentes pruebas o *tests* diseñados para verificar la estabilidad, seguridad y viabilidad de TutorIA.

El plan de validación se ha estructurado en cinco conceptos: la resiliencia de la infraestructura, la robustez del motor de compilación, el control de abusos y memoria, la evaluación del razonamiento del ecosistema de Inteligencia Artificial y finalmente, la validación mediante pruebas de usabilidad (*Beta Testing*). Para garantizar la mantenibilidad del proyecto, la mayor parte de estas comprobaciones se han automatizado empleando la librería `pytest` y las herramientas de simulación de peticiones nativas de FastAPI (`TestClient`), dividiendo los tests en archivos específicos (`test_compile.py`, `test_archivos.py`, `test_ia.py`, `test_terminal.py` y `test_api.py`).

5.2. Validación de la Infraestructura de Ejecución y Aislamiento

El núcleo crítico de TutorIA es su capacidad para manejar contenedores virtuales precargados (*Warm Pool*) que compilan y ejecutan el código de los estudiantes en tiempo real. La evaluación de este componente prioriza el encapsulamiento de amenazas y la preservación de la estabilidad del servidor.

5.2.1. Aislamiento de Red y Restricciones de Hardware

La instanciación de los entornos virtuales basados en Ubuntu 24.04 fue evaluada para garantizar que actúan como *sandboxes* herméticos. El gestor `WarmPoolManager` aplica políticas restrictivas de hardware a cada conte-

nedor instanciado, limitando la memoria RAM y la paginación a 256 MB (`--memory 256m`, `--memory-swap 256m`), y el procesamiento a medio núcleo de CPU (`--cpus 0.5`).

Para prevenir vulnerabilidades de escalada de privilegios, la imagen Docker configura al usuario estándar `alumno` y el orquestador anula todas las capacidades del kernel (`--cap-drop ALL`) junto con la prohibición de adquirir nuevos permisos (`--security-opt no-new-privileges`).

Listing 5.1: Fragmento de codigo para la configuracion de los contenedores

```
        cmd = [
                "docker", "run", "-d", "--rm",
                "--name", name,
                "--network", "none",
                "--memory", "256m",
                "--memory-swap", "256m",
                "--cpus", "0.5",
                "--pids-limit", "64",
                "--security-opt", "no-new-privileges",
                "--cap-drop", "ALL",
                "tutor-ugr-image:latest",
                "tail", "-f", "/dev/null"
            ]
```

Para certificar la desconexión total del exterior, el sistema levanta los contenedores con la directiva `--network none`. La automatización de esta comprobación se encuentra en el archivo `test_compile.py` mediante la función `test_seguridad_aislamiento_red`. Este test inyecta código C++ que intenta realizar una petición HTTP.

Listing 5.2: Verificación automatizada del aislamiento de red en contenedores

```
def test_seguridad_aislamiento_red():
    with TestClient(app) as client:
        payload = {
            "archivos": {
                "main.cpp": "#include <cstdlib>\nint main() { system
                    (\"curl http://google.com\"); return 0; }"
            }
        }
        response = client.post("/api/execute", json=payload)

        # Validar que no hay salida a Internet
        assert "html" not in response.json()["output"].lower()
```

5.2.2. Estabilidad del Despliegue en la Nube

Durante las pruebas en la infraestructura de Google Cloud Platform, se detectó que la instalación de paquetería APT durante la construcción del `Dockerfile` provocaba caídas por agotamiento de memoria (*Out Of Me-*

mory). Esto se solucionó implementando un archivo *Swap* de 2 GB en el servidor. También, para reducir los errores de tiempo de espera (*timeout*) al sincronizar repositorios, se validó la inclusión de la directiva `Acquire::ForceIPv4 "true"` en el sistema de paquetes del contenedor, forzando la resolución de dominios mediante IPv4 y garantizando reconstrucciones sin fallos de red.

5.3. Pruebas de Integración en el Motor de Ejecución

El orquestador de ejecución (`compiler.py`) es el encargado de traducir la entrada del usuario en órdenes directas para el entorno aislado. Las pruebas en este módulo certifican la heurística de compilación y la gestión del ciclo de vida de los archivos.

5.3.1. Compilación Multi-Archivo

Se diseñaron pruebas de integración dentro de `test_compile.py` para asegurar que el sistema adapta su comportamiento de forma inteligente según la estructura del proyecto enviado. El ensayo `test_compilar_con_makefile` verifica que, si el sistema detecta la existencia de un fichero `Makefile`, suspende la compilación predeterminada con `g++` y delega a la herramienta `make`, buscando después el binario en los directorios de salida (`./main`, `./dist/main`, `./bin/main`).

Listing 5.3: Prueba de integración de compilación automatizada mediante Makefile

```
def test_compilar_con_makefile():
    with TestClient(app) as client:
        payload = {
            "archivos": {
                "main.cpp": "#include <iostream>\nint main() { std::
                    cout << \"Make OK\"; return 0; }",
                "Makefile": "all:\n\tg++ main.cpp -o main"
            }
        }
        response = client.post("/api/execute", json=payload)

        # Verificar que usa el Makefile
        assert "Make OK" in response.json()["output"]
```

5.3.2. Detección de Bucles Infinitos y Limpieza de Recursos

Para evitar que el servidor se sature o se quede colgado por culpa de procesos bloqueados, el comando de ejecución envuelve el programa resultante

con la utilidad `timeout 10`. Esta medida de seguridad se comprueba en el archivo `test_compile.py` mediante la función `test_seguridad_bucle_infinito`. Aquí se inyecta un bucle `while(true)` y se verifica que el sistema operativo fuerza el cierre del contenedor a los 60 segundos, devolviendo el control al servidor.

Listing 5.4: Prueba de integración de apertura y cierre de la conexión

```
1
2 def test_websocket_terminal_conexion():
3     # Comprobamos que el tunel se abre sin crashear y lo cerramos de
          inmediato
4     # para que no se quede colgado esperando a que Docker arranque.
5     with client.websocket_connect("/ws/terminal") as websocket:
6         assert websocket is not None
7         websocket.close()
```

De igual forma, se verificó que la limpieza de recursos (*Garbage Collection*) funciona bien al usar WebSockets. En el archivo `test_terminal.py`, la prueba `test_websocket_terminal_conexion` asegura que la conexión a la terminal virtual (PTY) se abre y cierra correctamente. Esto garantiza que el parámetro `--rm` de Docker borre el contenedor en cuanto el alumno abandona la sesión, evitando que se acumulen procesos zombis que consumirían memoria a lo largo del tiempo.

Listing 5.5: Prueba de integración de eguridad para bucles infinitos

```
1 # Ubicacion: test_compile.py
2 def test_seguridad_bucle_infinito():
3     with TestClient(app) as client:
4         payload = {"archivos": {"main.cpp": "int main() { while(true)
              {} return 0; }"}}
5         response = client.post("/api/execute", json=payload)
6         # El sistema deberia devolver un mensaje de tiempo excedido
7         assert response.json()["isError"] == True
```

5.4. Seguridad Perimetral, Memoria y Control de Abusos

Esta tercera fase de pruebas se centra en la API, con el objetivo de evitar un mal uso de los servicios y asegurar que no se desborde la memoria del sistema.

5.4.1. Prevención de Spam y Limitación de Peticiones (Rate Limiting)

El uso del modelo de lenguaje supone un coste económico por cada *token* procesado. Para proteger la plataforma contra ataques de denegación de servicio (DoS) o el envío abusivo de mensajes, el *endpoint* del chat se ha limitado usando el algoritmo *Token Bucket* de la librería `SlowAPI`.

Esta restricción se verifica en el archivo `test_ia.py` mediante la prueba `test_seguridad_rate_limiting_chat`, la cual simula el comportamiento de un usuario intentando saltarse el límite de mensajes.

Listing 5.6: Prueba para comprobar el límite de peticiones del chat

```
# test_ia.py
def test_seguridad_rate_limiting_chat():
    payload_valido = {
        "historial": [{"role": "user", "content": "Spam test"}],
        "archivos": {}, "asignatura": "cpp", "terminal_context": ""
    }

    # Hacemos 5 peticiones (el limite)
    for _ in range(5):
        client.post("/api/chat", json=payload_valido)

    # La sexta deberia fallar con error 429
    response_bloqueada = client.post("/api/chat", json=payload_valido)
    assert response_bloqueada.status_code == 429
```

5.4.2. Control de Memoria RAM en la Subida de Archivos

El diseño del proyecto incluye la base para procesar documentos teóricos mediante la técnica RAG (Generación Aumentada por Recuperación). Aunque la interfaz visual para que los alumnos suban archivos en PDF se deja como trabajo futuro, la seguridad del *backend* correspondiente ya está implementada y probada en el módulo `test_archivos.py`.

El test `test_seguridad_limite_tamano_pdf` simula la subida de un archivo pesado (11 MB) creado directamente en memoria. La prueba confirma que FastAPI corta la conexión al instante y devuelve un error HTTP 413 (*Payload Too Large*), evitando así que la RAM del servidor colapse. Además, la prueba `test_upload_pdf_endpoint_sin_cabecera` asegura que el sistema rechaza cualquier petición a estos *endpoints* si no lleva un identificador de sesión válido.

Listing 5.7: Prueba para comprobar la subida de archivos PDF

```
def test_seguridad_limite_tamano_pdf():
    # Simulamos un "PDF Bomba" de 11 MB creado virtualmente en la
        memoria
```

```
    archivo_pesado_bytes = b"0" * (11 * 1024 * 1024)
    archivo_pesado = ("pesado.pdf", archivo_pesado_bytes, "application
        /pdf")

    response = client.post(
        "/upload-pdf",
        files={"file": archivo_pesado},
        headers={"x-session-id": "test_session_123"}
    )

    # El servidor debe rechazarlo inmediatamente con un codigo 413 (
        Payload Too Large)
    assert response.status_code == 413
    assert "demasiado grande" in response.json()["error"].lower()
```

5.5. Evaluación del Subsistema de Inteligencia Artificial

Las pruebas de la Inteligencia Artificial sirven para comprobar si el modelo es capaz de razonar correctamente y entender los proyectos de programación de los alumnos.

5.5.1. Detección de Errores Lógicos en Proyectos Reales

Se preparó un test en el script `test_api.py` para evaluar cómo se comporta la IA frente a errores de lógica, ya que suelen ser más difíciles de encontrar que los típicos fallos de sintaxis que detecta el compilador. Para ello, se mandó un proyecto dividido en varios archivos (`main.cpp`, `utils.h`, `utils.cpp`) introduciendo un fallo a propósito: la función `calcularDoble(int n)` multiplicaba por 3 en lugar de por 2.

Los resultados de este test confirmaron que la IA es capaz de mantener el hilo de la conversación, leer todo el árbol de archivos y encontrar el fallo matemático en el archivo secundario, dando al alumno las pistas necesarias para corregirlo.

Listing 5.8: Prueba para comprobar la respuesta de la IA

```
# Simulamos el payload que enviaria el front
payload = {
    "mensajes": [
        {
            "role": "user",
            "content": "Hola, he separado mi codigo en varios archivos
                . Intento calcular el doble de 5, pero el programa
                imprime 15 en lugar de 10. Por que falla mi logica"
        }
    ],
    "archivos": {
        "main.cpp": """
```

```
            #include <iostream>
            #include "utils.h"

            using namespace std;

            int main() {
                cout << "El doble de 5 es: " << calcularDoble(5) <<
                    endl;
                return 0;
            }
            """,
                    "utils.h": """
            #ifndef UTILS_H
            #define UTILS_H

            int calcularDoble(int n);

            #endif
            """,
                    "utils.cpp": """
            #include "utils.h"

            int calcularDoble(int n) {
                return n * 3;
            }
            """
    }
}
```

5.5.2. Contexto de la Terminal y Peticiones Canceladas

En el archivo `test_ia.py`, la prueba `test_chat_contexto_linux` comprobó que el modelo realmente detecta lo que ocurre en la terminal del alumno. Al pasarle un contexto simulado (`üsuario@linux:~/TFG$"`), la IA supo recomendar el comando `ls`, demostrando que el sistema inyecta bien el estado de la consola en la consulta.

Listing 5.9: Prueba para comprobar la respuesta de la IA con comando de terminal

```
def test_chat_contexto_linux():
    payload = {
        "historial": [{"role": "user", "content": "Dime un comando
            para listar archivos"}],
        "archivos": {}, "asignatura": "linux", "terminal_context": "
            usuario@linux:~/TFG$"
    }
    response = client.post("/api/chat", json=payload)
    respuesta = response.json()["reply"].lower()
    assert "ls" in respuesta or "comando" in respuesta
```

Por otro lado, se comprobó qué ocurre si un usuario cierra el navegador de golpe. En ese caso, FastAPI detecta el corte y lanza el error `asyncio.CancelledError`. El sistema captura este evento y frena la con-

sulta a la IA, lo que es vital para no gastar *tokens* innecesariamente si el alumno ya se ha desconectado. Se realizaron pruebas cerrando de forma abrupta la pestaña del navegador o recargando la página web mientras el servidor aún estaba esperando la respuesta de la API. Al romperse el *socket* TCP de la conexión del cliente, el servidor (Uvicorn) envía una señal de interrupción al hilo de ejecución de FastAPI, lo que desencadena una excepción `asyncio.CancelledError`.

Se verificó mediante los registros del sistema (*logs*) que el bloque `try except` implementado en el controlador captura correctamente este evento, mostrando la alerta de cancelación y abortando de inmediato el proceso en segundo plano. Esta validación demuestra la resiliencia del *backend*, siendo vital para no consumir ancho de banda ni gastar *tokens* de facturación de forma innecesaria en alumno que ya han abandonado la plataforma.

5.6. Pruebas con Usuarios Reales (Beta Testing)

Como parte final de la validación, se hizo una prueba práctica con un grupo de estudiantes del Grado en Ingeniería Informática. El objetivo era ver si la plataforma era intuitiva y si el enfoque de "tutor socrático" realmente ayudaba a resolver una práctica de C++.

5.6.1. Metodología de la Evaluación

El grupo de pruebas estuvo compuesto por 4 alumnos matriculados en asignaturas de programación. A cada participante se le proporcionó en el entorno aislado un fallo lógico predefinido en un programa escrito en C++. La idea principal de la prueba fue que debían encontrar y corregir el error interactuando exclusivamente con el asistente, sin recurrir a motores de búsqueda externos ni a documentación ajena a la plataforma.

Durante las sesiones, se observó el comportamiento de los estudiantes frente a los errores del compilador. Al finalizar la práctica, se llevó a cabo una breve entrevista para recoger sus impresiones sobre la usabilidad de la herramienta y la utilidad del enfoque implementado por la IA.

5.6.2. Resultados y Experiencia de Usuario

El análisis de las entrevistas y el uso de la plataforma nos dio conclusiones muy positivas sobre la viabilidad del proyecto en un entorno académico real:

- **Eficacia del método socrático:** Los alumnos valoraron especialmente que la Inteligencia Artificial se negara a proporcionar bloques

de código resuelto. Aunque en los primeros momentos algunos estudiantes mostraron cierta frustración por no obtener una respuesta directa, al finalizar la prueba reconocieron que las preguntas guiadas del tutor les obligaron a comprender realmente la raíz del fallo (como una mala gestión de punteros), evitando la dinámica tradicional de copiar y pegar sin asimilar los conceptos.

- **Comodidad y reducción de distracciones:** La integración del editor de código, la terminal interactiva y el chat en un único *dashboard* fue el aspecto técnico más destacado a nivel de interfaz. Los participantes señalaron que esta unificación disminuye radicalmente la carga visual y evita las distracciones que supone cambiar constantemente entre el IDE de escritorio, la consola y el navegador web.
- **Conocimiento automático del entorno:** Los estudiantes destacaron la eficacia que aporta el sistema al conocer de antemano el fichero que están editando y el error exacto que acaba de arrojar la consola. Al no tener que redactar explicaciones previas ni copiar manualmente la traza de error (*stack trace*) en el chat, la formulación de dudas resultó ser mucho más natural y directa.
- **Latencia y límites de interacción:** Varios usuarios notaron que el chat tardaba demasiado tiempo en procesar las respuestas y, en pruebas intensivas, las conversaciones se bloqueaban temporalmente. Al depender de servicios de terceros en sus capas gratuitas, era un escenario contemplado desde el diseño inicial.
- **Inconsistencias puntuales :** En algunos ejercicios o preguntas relacionadas con el temario oficial, los alumnos detectaron que el tutor llegaba a "alucinar", perdiendo el hilo del problema o dando pistas que resultaban confusas, incluso añadiendo links erróneos a "temario oficial".

5.7. Análisis de Casos: Éxitos y Alucinaciones

En esta sección se evalúa el comportamiento real del sistema mediante distintos casos de uso, documentando tanto los escenarios donde la arquitectura funciona según lo esperado (éxitos) como aquellos donde surgen problemas técnicos o comportamientos no deseados del modelo de lenguaje (alucinaciones y errores).

5.7.1. Casos de Éxito: Recuperación Precisa y Ejecución Correcta

El sistema demuestra una alta eficacia cuando el agente de IA utiliza correctamente la herramienta de recuperación de contexto (RAG). Como se observa en la Figura 5.1, ante una consulta sobre la diferencia entre `new` y `delete` en C++, el tutor socrático logra identificar no solo el tema, sino la ubicación exacta ("Tema 01 – Punteros y Memoria Dinámica.pdf – Diapositiva 50"). La respuesta se estructura de forma aceptable, combinando explicaciones teóricas y ejemplos de código.

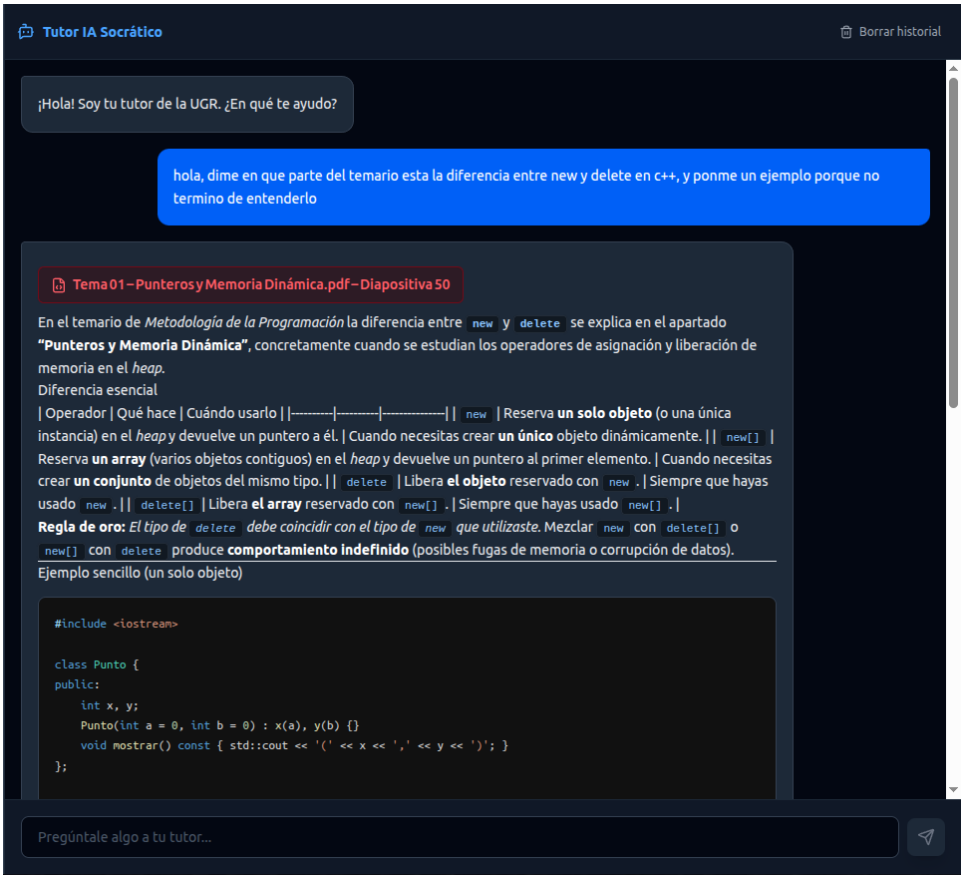


Figura 5.1: Respuesta exitosa del LLM

Este éxito se traslada también al entorno de ejecución con la prueba de compilación correcta (Figura 5.2), donde se evidencia que el código C++ es procesado adecuadamente por el contenedor aislado. La terminal integrada refleja la ejecución exitosa y la salida estándar, confirmando que la conexión entre el frontend y el servidor de ejecución opera correctamente.

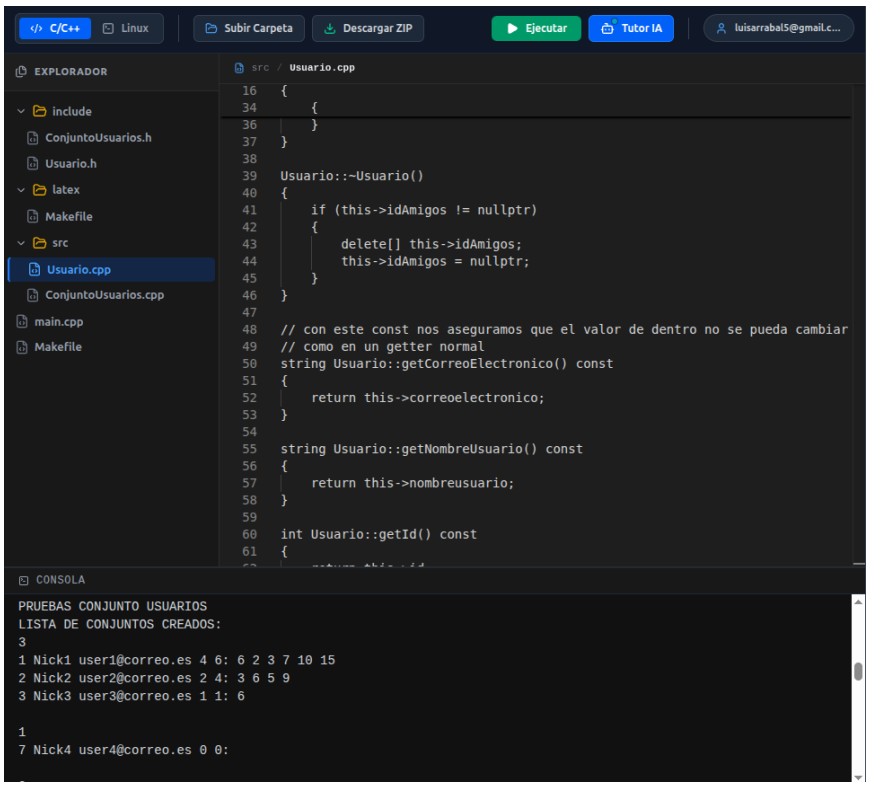


Figura 5.2: Compilación exitosa del código

5.7.2. Manejo de Errores

El entorno de desarrollo integrado en la aplicación cumple su función de informar al usuario cuando el código es defectuoso. En el caso de la compilación errónea (Figura 5.3), el sistema captura y muestra con precisión el error del compilador `g++` (en este caso, una variable `j` no declarada en el ámbito de `ConjuntoUsuarios.cpp`). Esto demuestra que el flujo de retroalimentación de errores desde el contenedor Docker hasta la interfaz web es fiable.

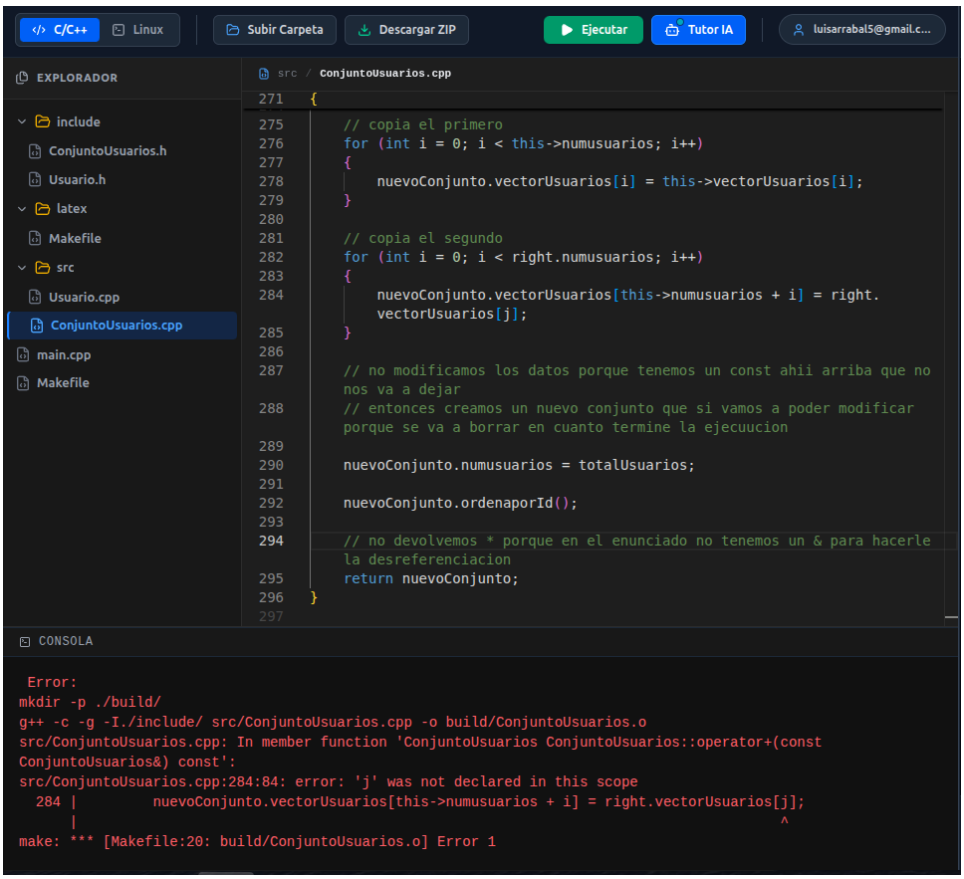


Figura 5.3: Captura y visualización de error de compilación.

5.7.3. Alucinaciones y Errores de Formato

A pesar de los buenos resultados generales, el uso de LLMs conlleva el riesgo de respuestas inesperadas o problemas de renderizado. En la Figura 5.4, se aprecia un comportamiento mixto. Inicialmente, la IA no acierta al ubicar la duda en el "Tema 3: Memoria Dinámica", puesto que pertenece al Tema 1, sin embargo, el enlace al contenido oficial es correcto y el contenido de teoría también. Posteriormente, ante una siguiente pregunta, el modelo sufre una ligera "alucinación" de formato, en lugar de devolver la información estructurada el modelo inyecta directamente etiquetas HTML (`<a href="https://apuntes.ugr.es...">Aquí</a>`).



Figura 5.4: Comportamiento mixto: acierto en contexto y error de formato HTML.

Este y otros posibles errores futuros, exponen posible información sensible al usuario final. Esto enfatiza en la necesidad de implementar filtros de post-procesamiento más estrictos en el backend antes de enviar la respuesta final por el WebSocket, para asegurar que cualquier enlace o referencia generada por la IA se formatee estrictamente bajo los estándares visuales de la aplicación.

5.7.4. Mejoras Detectadas para el Futuro

Aunque el resultado general de las pruebas fue positivo, la evaluación con los compañeros permitió identificar algunas limitaciones y aspectos a mejorar. Estas sugerencias han servido de base para definir las funcionalidades que deberían incluirse en las próximas versiones de la herramienta:

- **Interfaz de Subida de Documentos (RAG):** La lógica interna y la seguridad para procesar archivos PDF ya están terminadas en el *backend*, pero todavía falta diseñar la interfaz visual. El objetivo es añadir una pantalla donde el alumno pueda subir sus apuntes, con una barra de carga que muestre el progreso mientras el sistema procesa el texto para que la IA lo pueda consultar posteriormente.

- **Acciones Rápidas en el Editor:** Para que el uso sea más fluido, se sugirió poner botones pequeños cerca del código con opciones típicas (como "Explícame este error" o "Dame una pista"). De esta forma, el estudiante no tiene que escribir a mano la misma duda cada vez que el código no compila.
- **Panel de Analíticas para Profesores (*Dashboard*):** Desde el punto de vista del docente, sería muy útil contar con un panel que recoja estadísticas sobre los errores más comunes en C++ o las dudas que más se repiten en la plataforma. Esto ayudaría a los profesores a detectar qué temas cuestan más y adaptar así las clases teóricas.
- **Juez Virtual (Evaluación Automatizada):** Varios alumnos comentaron que la plataforma sería ideal para hacer relaciones de problemas. La idea es crear un sistema tipo *HackerRank*, donde el código se evalúe automáticamente pasando unos casos de prueba ocultos (*test cases*). Lo interesante de integrar esto con la IA es que, si el código falla en un caso extremo, el tutor virtual podría intervenir directamente para hacer pensar al alumno sobre el fallo lógico, en lugar de limitarse a marcar la solución como incorrecta.

Capítulo 6

Conclusiones y Trabajos Futuros

6.1. Conclusiones

El presente Trabajo de Fin de Grado recoge el diseño y la implementación de la plataforma TutorIA, logrando los objetivos generales y específicos propuestos al inicio del proyecto. Mediante el uso de tecnologías web, arquitecturas multi-agente y modelos de lenguaje (LLM), se ha desarrollado una herramienta de apoyo al estudio que busca cubrir las limitaciones pedagógicas de los asistentes de programación actuales.

De acuerdo con los objetivos específicos definidos en el Capítulo 1, los principales resultados obtenidos son:

- **Desarrollo de la plataforma y tutoría socrática:** Se ha construido una aplicación web que integra un editor de código, una terminal y un chat. El flujo de ejecución, controlado por LangGraph, fuerza la aplicación del método socrático mediante ingeniería de instrucciones (*Prompt Engineering*). Esto permite guiar al alumno con preguntas, en lugar de generar el código directamente, favoreciendo así su razonamiento.
- **Integración del motor RAG:** La inclusión de un sistema de Generación Aumentada por Recuperación, conectado a una base de datos vectorial (Supabase), restringe el contexto del modelo de lenguaje. Al obligar a la IA a basar sus respuestas en el material docente oficial de la asignatura, se limita de manera considerable la aparición de explicaciones incorrectas o fuera de temario (alucinaciones).
- **Aislamiento y ejecución de código:** Se ha integrado un entorno de pruebas remoto (*sandboxing*) basado en contenedores Docker. El uso de un *Warm Pool* y conexiones WebSocket hace posible que el estudiante compile código C++ y ejecute comandos en Linux de forma interactiva, manteniendo el servidor protegido frente a código malicioso o errores de ejecución.
- **Perfilado asíncrono del estudiante:** El diseño multi-agente inclu-

ye un segundo agente analista que funciona en segundo plano. Este componente evalúa las conversaciones para extraer las debilidades y fortalezas del alumno, guardando esta información en la base de datos sin afectar a la fluidez ni a los tiempos de respuesta del chat principal.

- **Pruebas y validación del sistema:** Se ha realizado una evaluación exhaustiva de la plataforma mediante pruebas funcionales y casos de uso prácticos. Esta validación ha verificado la estabilidad de la infraestructura en la nube y ha demostrado la utilidad real de la herramienta para guiar de manera precisa y segura el aprendizaje de los alumnos.

En conclusión, la combinación de modelos generativos con el protocolo MCP (*Model Context Protocol*), bases de datos vectoriales y ejecución aislada supone una solución técnica viable para el entorno académico. TutorIA propone un enfoque alternativo y más controlado para la integración de la IA en la universidad, centrando el uso de la tecnología en el apoyo formativo del alumno en lugar de en la mera resolución de problemas.

6.2. Limitaciones y Desafíos Técnicos

A pesar de que la arquitectura diseñada ha superado satisfactoriamente los casos de prueba y ha validado su utilidad pedagógica, la naturaleza probabilística de los modelos de lenguaje y las restricciones de la infraestructura actual imponen ciertas limitaciones operativas que deben tenerse en cuenta:

- **Latencia de Arranque (*Cold Start*) e Infraestructura:** El aislamiento mediante contenedores Docker garantiza la seguridad frente a código malicioso, pero consume recursos considerables a nivel de sistema operativo. En la infraestructura actual, alojada en una Máquina Virtual de bajo coste en Google Cloud (`e2-micro`), la memoria RAM y el procesamiento son restrictivos. Aunque el gestor *Warm Pool* mitiga los tiempos de espera al precalentar entornos virtualizados, un pico elevado (por ejemplo, decenas de alumnos compilando simultáneamente) agotaría los recursos de memoria, elevando el tiempo de respuesta de forma significativa y provocando cuellos de botella en la terminal interactiva.
- **Cuotas de Uso y Límites de Tasa (*Rate Limits*):** El proyecto se ha desarrollado aprovechando las capas gratuitas (*Free Tiers*) de las APIs de Groq y Gemini. Estas plataformas imponen límites de peticiones por minuto para evitar abusos. En un escenario de uso intensivo en el aula, el sistema se enfrentaría a rechazos de conexión de tipo HTTP 429 (*Too Many Requests*), lo que paralizaría temporalmente la

capacidad del agente para responder o vectorizar texto, o incluso de *tokens* agotados para el modelo.

- **Umbral de Alucinación en Lógica Abstracta:** Si bien el motor de Generación Aumentada por Recuperación (RAG) elimina casi por completo las alucinaciones teóricas al forzar a la IA a basarse en los apuntes oficiales de la UGR, siguen los desafíos en la revisión de código complejo. Si el error del estudiante involucra conceptos de algoritmia muy abstractos o violaciones de memoria profunda con punteros que no están explícitamente detallados en el temario inyectado, el modelo puede perder el contexto espacial del problema y ofrecer sugerencias de depuración genéricas, imprecisas o cíclicas, incluso al tratarse de modelos genéricos, puede dar lugar a que sufra de alucinaciones simplemente manteniendo una conversación sobre programación enfocada al temario oficial.

6.3. Dificultades Encontradas

El desarrollo de TutorIA ha supuesto un reto de ingeniería de software considerable, integrando tecnologías que abarcan desde el bajo nivel del sistema operativo hasta la orquestación de modelos de lenguaje en la nube. A lo largo del ciclo de vida del proyecto, se presentaron diversas dificultades técnicas que exigieron un rediseño de las estrategias iniciales:

- **Aislamiento y Permisos en Docker:** La configuración del *sandboxing* presentó serios problemas de despliegue en las primeras versiones. Al intentar limitar los privilegios de los contenedores (`--cap-drop ALL`), se descubrió que ciertas operaciones de compilación de C++ fallaban al carecer de acceso a recursos específicos del kernel. Fue necesario un proceso de depuración sobre las políticas de seguridad (`--security-opt`) hasta encontrar el equilibrio exacto entre seguridad y funcionalidad.
- **Ingeniería de Instrucciones (*Prompt Engineering*) y Comportamiento del Modelo:** En el ámbito de la Inteligencia Artificial, el mayor desafío no fue la conexión técnica con las APIs de Groq o Gemini, sino "controlar" el comportamiento del modelo. Los Grandes Modelos de Lenguaje están entrenados para proporcionar respuestas directas a cualquier problema, y diseñar un *System Prompt* socrático que lograse contenerlo, forzando a la IA a hacer preguntas sin revelar el código, requirió decenas de iteraciones empíricas, ajustes de temperatura y pruebas de inyección de contexto.

- **Gestión de Memoria en la Nube:** Durante el despliegue en Google Cloud Platform, la máquina virtual de capa gratuita experimentó caídas por agotamiento de memoria (*Out Of Memory*) durante la reconstrucción de imágenes. Esto obligó a optimizar el flujo de CI/CD, añadir memoria de intercambio (*Swap*) y programar rutinas de limpieza automáticas (*Garbage Collection*) para eliminar contenedores zombis y liberar recursos.
- **Limitaciones de Infraestructura:** El paso desde el entorno de desarrollo local al despliegue en la nube resultó ser la fase más crítica y compleja del proyecto. Las instancias de bajo coste en la nube carecen de la memoria RAM necesaria para ejecutar tareas de construcción de contenedores (`docker build`). Esto causaba que el servidor se congelara tras intentar compilar librerías de Ubuntu, lo que obligó a reestructurar la configuración del sistema operativo anfitrión inyectando archivos de paginación (*Swap*) de 2 GB y forzando arquitecturas de red IPv4 para evitar *timeouts* persistentes en los repositorios de APT.
- **Políticas de Seguridad en Navegadores (*Mixed Content*):** Otro de los grandes retos fue el rechazo de las conexiones por parte del *frontend*. Al alojar la interfaz en Vercel (bajo protocolo HTTPS) e intentar abrir un WebSocket hacia la IP pública de la máquina de Google Cloud (HTTP), los navegadores bloqueaban la comunicación de la terminal. Solucionar este conflicto requirió el diseño de un túnel inverso persistente con certificados SSL automáticos mediante Ngrok y la gestión de este proceso junto con la API empleando el demonio PM2(facilita ejecutar la apliación en segundo plano de manera continua y su despliegue).

6.4. Trabajos Futuros

A partir de la base tecnológica construida en este trabajo, se abren varias vías de ampliación para convertir la plataforma en una herramienta de apoyo consolidada y adaptable a distintas asignaturas de la Universidad de Granada. Entre las principales vías de trabajo futuro se proponen las siguientes:

- **Panel de Control para Docentes (*Dashboard*):** Se plantea el desarrollo de una interfaz específica para el profesorado que muestre métricas y estadísticas agregadas de uso. Analizando de forma anónima los errores de compilación más repetidos y las dudas más frecuentes de la clase, los docentes podrían identificar qué conceptos teóricos necesitan mayor refuerzo en las horas de teoría.

- **Soporte para Nuevos Lenguajes de Programación:** Aunque el sistema actual está diseñado y probado en C++, y la terminal en comandos *Linux*, la arquitectura basada en imágenes Docker permite escalar la herramienta sin modificar el núcleo de la aplicación. El siguiente paso lógico sería crear configuraciones para soportar lenguajes como Java o Python, ampliando su utilidad a otras asignaturas del Grado como Programación y Diseño Orientado a Objetos.

- **Integración de Herramientas de Análisis:** Para mejorar la retroalimentación, se propone la integración de herramientas como *Valgrind*. Con esta mejora, el sistema detectaría automáticamente fugas de memoria o malos usos de punteros y le pasaría esa información a la IA, permitiendo que el tutor ofrezca explicaciones detalladas sobre la gestión de memoria antes de que el programa falle por completo.

- **Persistencia y Gestión de Usuarios:** Resulta fundamental implementar un sistema robusto de autenticación y registro. Esta mejora de infraestructura permitiría a los estudiantes guardar el estado de su código en la nube, conservar el historial de conversaciones con el tutor virtual y recuperar su entorno de trabajo desde cualquier dispositivo en distintas sesiones, sentando además las bases técnicas para realizar un seguimiento a largo plazo.

- **Evolución del Ecosistema Multi-Agente y Perfilado Cognitivo:** Se propone refinar los canales de comunicación y la sincronización entre el agente analista y el tutor, mediante la incorporación de sistemas de memoria a largo plazo más complejos (como grafos de conocimiento), el agente analista podría estructurar un modelo profundo del estudiante, registrando patrones de error recurrentes, carga cognitiva y estilo de aprendizaje. Esta base de conocimiento compartida y actualizada permitiría al tutor ajustar su estrategia pedagógica y sus pistas de forma hiper-personalizada en cada interacción.

- **Plataforma de Retos (Juez Virtual):** Tal y como sugirieron los alumnos durante la fase de validación, se podría implementar un módulo de evaluación automatizada mediante casos de prueba (*test cases*). Esto transformaría la herramienta en un entorno guiado donde el agente de IA asistiría al estudiante si su código pasa los casos básicos, pero falla en las condiciones límite (*edge cases*).

- **Migración a la Infraestructura Institucional:** Para solventar las limitaciones de recursos del uso de una máquina virtual de bajo coste en la nube (Google Cloud Platform), se propone desplegar el entorno de ejecución (*backend* y el gestor *Warm Pool* de Docker) directamente en los servidores o clústeres de la Universidad de Granada. Esto permitiría escalar el sistema para soportar a cientos de alumnos concurrentes con

un hardware mucho más potente, eliminando la latencia de arranque y los costes operativos mensuales de infraestructura.

- **Despliegue de Modelos de Lenguaje Locales:** En relación con las consideraciones éticas y de privacidad, el siguiente paso consistiría en prescindir de las APIs de terceros (Groq y Gemini). Utilizando la infraestructura de cómputo de la universidad, se podrían alojar y ejecutar modelos fundacionales de código abierto (como Llama 3 o Mistral). Esta transición garantizaría el control absoluto de los datos de los estudiantes, el cumplimiento estricto del RGPD y la eliminación definitiva de los límites de tasa (*Rate Limits*) comerciales.

Bibliografía

[1] H. Goyal, R. Gupta, S. Sharma, K. Mittal, A. Mehra, J. Challa, and D. Kumar, "Sakshm ai: Impulsando la educación en programación asistida por ia para estudiantes de ingeniería en india a través de tutorías socráticas y retroalimentación integral," *ACM J. Comput. Sustain. Soc.*, vol. 4, apr 2026.

[2] S. Haroud and N. Saqri, "Generative ai in higher education: Teachers' and students' perspectives on support, replacement, and digital literacy," *Education Sciences*, vol. 15, no. 4, 2025.

[3] A. Mukherjee and S. Singh, "Leveraging ai chatbots for self-directed learning: A study of practices and drivers to adoption among college students," *Turkish Online Journal of Distance Education*, vol. 27, no. 1, p. 70–81, 2026.

[4] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, Mar. 2023.

[5] Y.-M. Yan, C.-Q. Chen, Y.-B. Hu, and X.-D. Ye, "Llm-based collaborative programming: impact on students' computational thinking and self-efficacy," *Humanities and Social Sciences Communications*, vol. 12, no. 1, p. 149, 2025.

[6] G. Jošt, V. Taneski, and S. Karakatič, "The impact of large language models on programming education and student learning outcomes," *Applied Sciences*, vol. 14, no. 10, 2024.

[7] M. B. Mutanga, J. Msane, T. N. Mndaweni, B. B. Hlongwane, and N. Z. Ngcobo, "Exploring the impact of llm prompting on students' learning," *Trends in Higher Education*, vol. 4, no. 3, 2025.

[8] F. Liu, Y. Liu, L. Shi, Z. Yang, L. Zhang, X. Lian, Z. Li, and Y. Ma, "Beyond functional correctness: Exploring hallucinations in llm-generated code," *IEEE Transactions on Software Engineering*, vol. 52, no. 3, pp. 1037–1055, 2026.

[9] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27730–27744, 2022.

[10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24824–24837, 2022.

[11] T. Şakar and H. Emekci, "Maximizing rag efficiency: A comparative analysis of rag methods," *Natural Language Processing*, vol. 31, no. 1, pp. 1–25, 2025.

[12] A. A. Kamalipour, S. Asadi, and M. M. Amiri Chimeh, "From vectors to knowledge graphs: A comprehensive analysis of modern retrieval-augmented generation architectures," *Computer Science Review*, vol. 61, p. 100925, 2026.

[13] F. R. Zagatti, G. Yuuji Shimizu, D. Lucrédio, and H. de Medeiros Caseli, "Investigating the relationship between text vectorization cosine similarity and classification performance," *IEEE Access*, vol. 13, pp. 137348–137363, 2025.

[14] L. Weng, "Llm powered autonomous agents." `https://lilianweng.github.io/posts/2023-06-23-agent/`, 2023. Referencia técnica estándar para la arquitectura de agentes autónomos y sistemas de orquestación de tareas.

[15] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation," 2023.

[16] J. Moura, "Crewai: Framework for orchestrating role-playing autonomous ai agents," 2025.

[17] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," 2024.

[18] Anthropic, "Model context protocol: Architecture," 2024. Accedido: 29 de mayo de 2026.

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[20] H. Chase, "Langchain." `https://github.com/langchain-ai/langchain`, 2022.

[21] Microsoft, "Semantic kernel: Integrate cutting-edge llm technology quickly and easily into your apps." `https://learn.microsoft.com/en-us/semantic-kernel/`, 2024. Accedido en 2026.

[22] Anthropic, "Introduction to model context protocol." https://anthropic.skilljar.com/introduction-to-model-context-protocol?next=%2Fintroduction-to-model-context-protocol%2F296690, 2025. Accedido: 23 de mayo de 2026.

[23] M. A. Jayanti and X. Y. Han, "Enhancing model context protocol (mcp) with context-aware server collaboration," 2026.

[24] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," *Commun. ACM*, vol. 68, p. 96–105, Jan. 2025.

[25] J. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather, "The robots are coming: Exploring the implications of openai codex on introductory programming," in *Proceedings of the 24th Australasian Computing Education Conference*, ACE '22, (New York, NY, USA), p. 10–19, Association for Computing Machinery, 2022.

[26] R. Liu, C. Zenke, C. Liu, A. Holmes, P. Thornton, and D. J. Malan, "Teaching cs50 with ai: Leveraging generative artificial intelligence in computer science education," in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, SIGCSE 2024, (New York, NY, USA), p. 750–756, Association for Computing Machinery, 2024.

[27] Meta Platforms, Inc., "React documentation." https://react.dev/, 2026. Accedido: 20-jun-2026.

[28] S. Ramírez, "Fastapi documentation." https://fastapi.tiangolo.com/, 2026. Accedido: 20-jun-2026.

[29] Supabase Inc., "Supabase documentation." https://supabase.com/docs, 2026. Accedido: 20-jun-2026.

[30] Docker Inc., "Docker overview: Containers and virtual machines," 2026. Accedido: 24 de mayo de 2026.

[31] Vercel, Inc., "Vercel Documentation." https://vercel.com/docs, 2026. Accessed: May 31, 2026.

[32] Google Cloud, "Google Cloud Platform Documentation." https://docs.cloud.google.com/docs, 2026. Accessed: May 31, 2026.

[33] Keymetrics, "Pm2: Advanced process manager." https://pm2.keymetrics.io/, 2026. Accedido: 31 de mayo de 2026.

[34] ngrok Inc., "ngrok: Secure tunnels to localhost." https://ngrok.com/docs, 2026. Accedido: 31 de mayo de 2026.

[35] V. Wang, F. Salim, and P. Moskovits, "The websocket protocol," in *The Definitive Guide to HTML5 WebSocket*, pp. 33–60, Springer, 2013.

[36] S. Ahmed and Q. Mahmood, "An authentication based scheme for applications using json web token," in *2019 22nd International Multitopic Conference (INMIC)*, pp. 1–6, 2019.

[37] GitHub, *GitHub Actions documentation*. GitHub, 2026. Consultado el 31 de mayo de 2026.

[38] Cburnett, "Artificial neural network." Wikimedia Commons, 2006. Accedido: 27 de mayo de 2026. Licencia: CC BY-SA 3.0.

[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[40] Docker Inc., "Docker overview: What is a container?," 2026. Accedido: 24 de mayo de 2026.